LongDS-Bench: On the Failure of Long-Horizon Agentic Data Analysis

Kewei Xu[1,3], **Xiaoben Lu**[1], **Shuofei Qiao**[1], **Zihan Ding**[1],
Haoming Xu[1], **Lei Liang**[2,3], **Ningyu Zhang**[1,3]*
[1]Zhejiang University, [2]Ant Group
[3]Zhejiang University - Ant Group Joint Laboratory of Knowledge Graph
{kewe1x,zhangningyu}@zju.edu.cn

Abstract

Real-world data analysis is inherently iterative, yet existing benchmarks mostly evaluate isolated or short interactive tasks, leaving agents' ability to track evolving analytical context over long horizons untested. We introduce LongDS, a benchmark for long-horizon, multi-turn data analysis where agents must maintain, update, restore, and compose evolving analytical states. LongDS comprises 68 tasks constructed from real-world Kaggle notebooks, spanning 2,225 turns across six domains including Geoscience, Business, and Education. Tasks are designed around state-evolution patterns (e.g., counterfactual perturbation, rollback, multi-state composition), with an average dependency span of 11.3 turns. Evaluating five state-of-the-art models, we find that the best model reaches only 48.45% average accuracy, performance drops nearly 47 points from early to late turns, and long-horizon errors account for 52%–69% of failures. Further analysis shows that additional agent steps do not necessarily improve performance, suggesting that the key bottleneck is maintaining a correct analytical state rather than increasing interaction budget. We release LongDS to support research on reliable long-horizon agentic data analysis[1].

1 Introduction

Large language model (LLM) agents are increasingly used for data analysis, where they write code and execute tools to analyze data and derive insights (Guo et al., 2024; Hong et al., 2024; Zhang et al., 2025b). However, real-world data analysis is rarely a sequence of independent, self-contained requests. Analytical workflows often unfold over extended persistent sessions, where scopes, metrics, assumptions, and intermediate results accumulate and shift across turns. Handling such workflows requires maintaining an evolving analytical state for interpreting and executing each request in context.

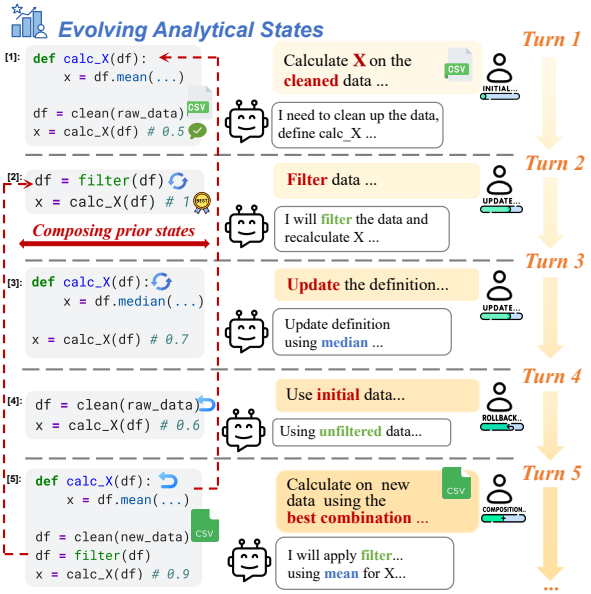


Figure 1: Multi-turn, long-horizon analytical state management in LongDS. Agents track evolving filters, definitions, and intermediate results to select the correct state for requests depending on prior turns.

Yet existing data analysis benchmarks provide limited evaluation of how agents manage analytical state over long horizons. Many benchmarks focus on independent tasks in resettable environments (Lai et al., 2023; Hu et al., 2024; Jing et al., 2025; Egg et al., 2025). Recent interactive benchmarks extend to multi-turn data analysis, but they often emphasize guided analysis completion, where the required operation is largely specified by the current turn (Dutta et al., 2025; Luo et al., 2026; Li et al., 2025). As a result, they leave open whether agents can manage evolving analytical states across long dependency chains, including updating states, applying local perturbations, rolling back to earlier states, and composing multiple states. A detailed comparison is provided in Table 1 and Appendix B.

To address this gap, we introduce **LongDS**, a benchmark for evaluating long-horizon agentic data analysis over evolving analytical states. Built from

* Corresponding Author.
[1]https://github.com/zjunlp/DataMind.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 13351–13383
October 24-29, 2026 ©2026 Association for Computational Linguistics

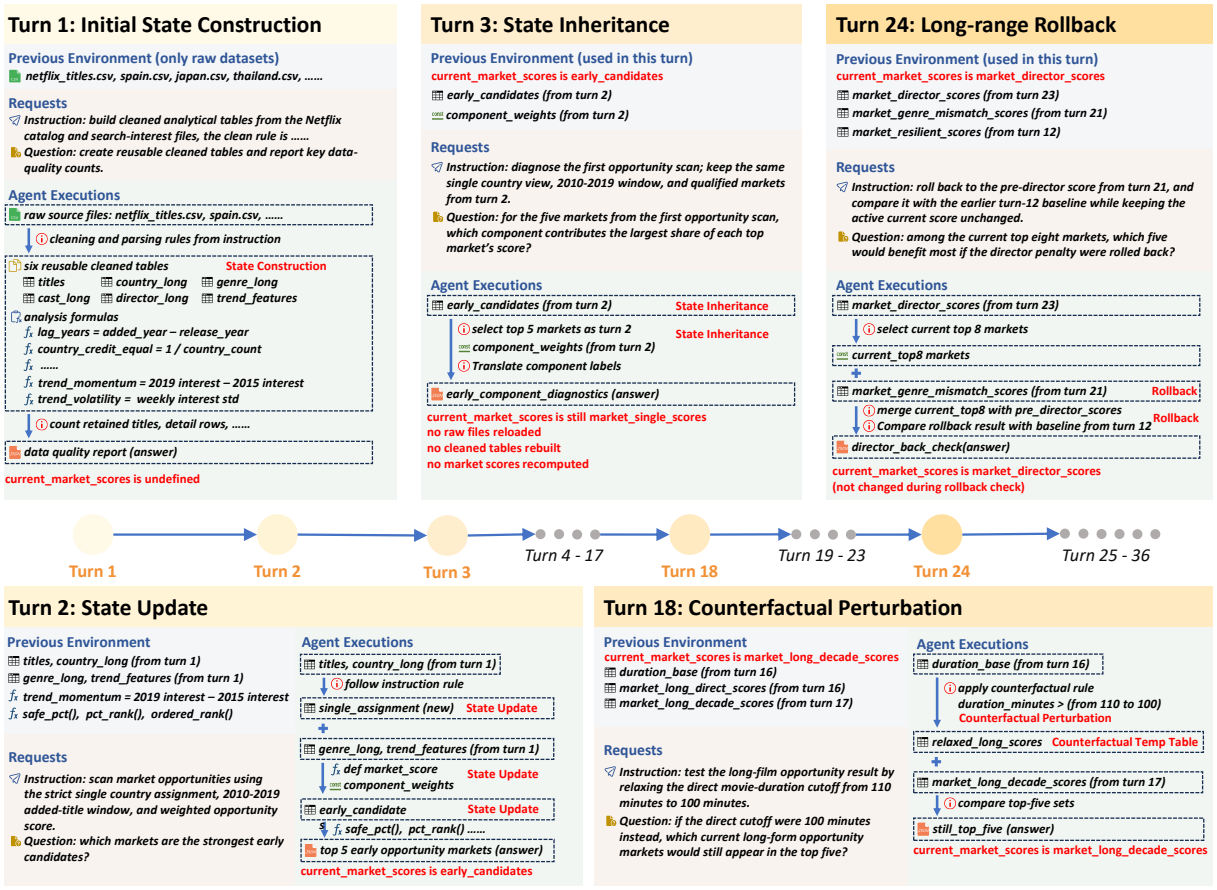


Figure 2: An example LongDS task illustrating five representative **state-evolution patterns** in a Netflix market-opportunity analysis spanning 36 turns. *Turn 1* constructs reusable analytical tables from raw files, establishing the **initial** analytical state. *Turn 2* builds on these tables to **update** the state with early market candidates. *Turn 3* **inherits** the candidates and component weights from Turn 2 to diagnose score contributors without recomputing the analysis. *Turn 18* inherits long-film scores from Turns 16–17 and applies a temporary **counterfactual perturbation** to the duration cutoff while preserving the default state. *Turn 24* uses the current top markets from Turn 23 but **rolls back** to the pre-penalty scores from Turn 12 to isolate the effect of the director-concentration penalty.

real-world Kaggle notebooks and datasets, LongDS converts realistic workflows into multi-turn tasks organized around state-evolution patterns (Table 3), with long-range turn dependencies. Figure 1 illustrates the benchmark setting, where each task unfolds as a persistent multi-turn session and subsequent requests depend on analytical states established or updated in prior turns. Comprising 68 tasks and 2,225 turns across six diverse application domains, including Geoscience, Business, and Education, LongDS provides a challenging testbed for long-horizon analytical state management, with an average dependency span of 11.3 turns.

Our experiments reveal that long-horizon analytical state management poses a major challenge for current agents. Across five state-of-the-art models, even the best model remains below 50% average accuracy on LongDS, with performance degrading sharply as interactions progress. Error analysis shows that most failures are long-horizon in nature, dominated by cascading and state-management errors rather than isolated coding or reasoning mistakes. Moreover, increasing the number of agent steps does not consistently improve accuracy, indicating that the main limitation lies in analytical state maintenance rather than interaction budget.

In summary, our contributions are threefold:

- We formulate long-horizon agentic data analysis as analytical state management, covering initial construction, inheritance, updates, counterfactual perturbations, rollbacks, and multi-state composition.
- We introduce LongDS, a realistic benchmark constructed from real-world workflows, comprising 68 tasks and 2,225 turns with long-range state dependencies.
- We provide a systematic evaluation of strong

Benchmark	Long-Horizon	State Evolution Pattern					# Tasks	Avg. Turns	Turn Purpose
		Init	Inh	Upd	CF	RB			
Single-Turn									
DABStep (Egg et al., 2025)	×	×	×	×	×	×	450	N/A	N/A
AgentDS (Luo et al., 2026)	◇	×	×	×	×	×	17	N/A	N/A
LongDA (Li et al., 2026b)	✓	×	×	×	×	×	505	N/A	N/A
Multi-Turn									
ConDABench (Dutta et al., 2025)	×	◇	◇	×	×	×	1,420	∼1.81	Task clarification
IDA-Bench (Li et al., 2025)	◇	◇	◇	◇	×	×	25	∼9.80	Task guidance
LongDS	✓	✓	✓	✓	✓	✓	**68**†	**∼32.7**	**New analytical request**

Table 1: Comparison of **LongDS** with existing data analysis benchmarks. **LongDS** targets the **long-horizon** management of **evolving analytical states** over extended multi-turn analysis. Init., Inh., Upd., CF, and RB denote initial construction, inheritance, update, counterfactual, and rollback, respectively. ◇ denotes implicit or partial coverage. † The 68 LongDS tasks span a total of **2,225** turns, each corresponding to a concrete data analysis request.

proprietary and open-source models, revealing substantial performance degradation over long trajectories and failures dominated by cascading and state-management errors.

2 Preliminary

A multi-turn data analysis task consists of a sequence of user requests over a collection of data files, carried out in a persistent executable environment such as a Jupyter notebook. Formally, a task is defined as:

$$\mathcal{T} = (\mathcal{D}, E_0, U)$$

where $\mathcal{D}$, E_0, and $U = (u_1, \ldots, u_T)$ denote the data files, the initial executable environment, and the sequence of user requests, respectively.

At turn t, an agent receives the current request u_t, interaction history $H_{<t}$, and current environment state E_{t-1}. Here, $H_{<t}$ denotes the interaction history before turn t, including previous user requests, agent responses, and executed analysis steps. The agent then performs analysis and returns a response y_t, resulting in an updated environment state E_t. Unlike isolated data analysis tasks, the environment is not reset between turns, allowing intermediate code states and results to persist.

The target response at turn t is determined by the current request u_t, the current environment state E_{t-1}, and the analytical context accumulated in $H_{<t}$, such as prior scopes, definitions, and assumptions. The goal of a multi-turn data analysis agent is to produce a sequence of target responses:

$$Y = (y_1, \ldots, y_T)$$

This setting captures long-horizon data analysis, where later requests may depend on analytical states that are inherited, revised, temporarily perturbed, or restored from many turns earlier.

3 LongDS Benchmark

3.1 Design Principles

LongDS evaluates data-analysis agents' ability to reason over **evolving analytical states** across long-horizon interactions, including scopes, definitions, assumptions, and intermediate results. A central challenge is that the valid analytical state is not static. Across a multi-turn trajectory, user requests may introduce new analytical objects, inherit existing ones, revise previous definitions, temporarily perturb assumptions, or restore earlier versions of the analysis. We therefore construct tasks around the state-evolution patterns summarized in Table 3.

These patterns differ in how they affect the active analytical state: **updates** overwrite the default state, **counterfactual perturbations** apply only locally, **rollbacks** answer the current request under an earlier anchored state, and **multi-state composition** requires combining multiple states. **Inheritance** is included for clarity, but it is treated as the default persistence mechanism rather than a separately annotated category in benchmark statistics. Figure 2 provides a representative example of such long-horizon state evolution in a 36-turn Netflix market-opportunity analysis task.

3.2 Task Curation

We construct LongDS from real-world data analysis notebooks to build long-horizon interactive tasks where later requests depend on analytical states established and revised across realistic workflows. The construction process consists of three steps as shown in Figure 3.

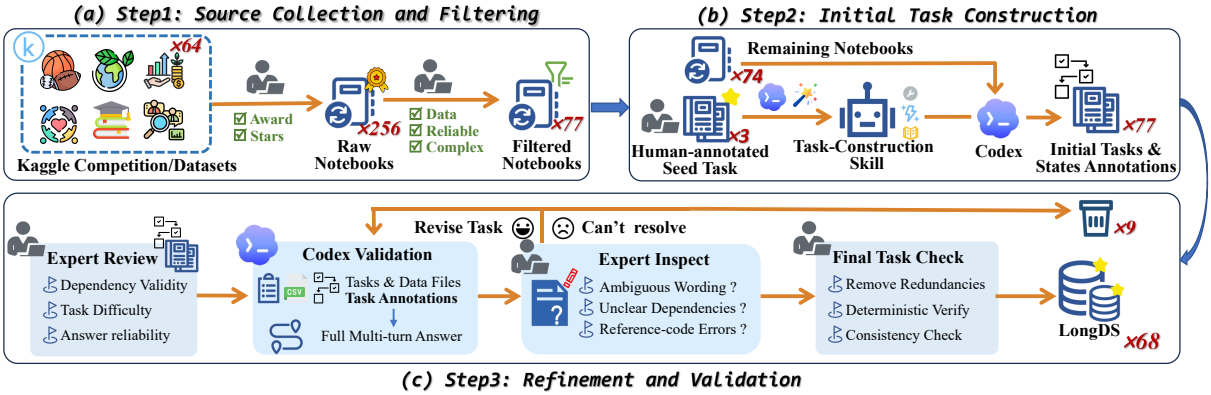


Figure 3: Task curation pipeline of LongDS: (a) source collection and filtering, (b) initial state-annotated task construction, and (c) refinement through expert review, Codex-based validation, and consistency checks.

Source Collection and Filtering. We curate raw notebooks from established Kaggle analytics competitions and highly upvoted public datasets. Unlike prediction-focused competitions, these sources capture full analysis workflows rather than isolated prediction tasks, providing realistic chains for constructing long-horizon tasks. This process yields an initial pool of 64 competitions and datasets. For each source, we select four notebooks from winning submissions or highly upvoted public notebooks, resulting in 256 high-quality raw notebooks spanning diverse domains.

We then manually execute and inspect each selected notebook, filtering them based on data accessibility, execution reliability, and computational feasibility. We fix minor execution issues or obvious code errors when they do not alter the intended analysis, and remove notebooks with insufficient analytical depth for long-horizon task construction. After filtering and repair, we retain 36 competitions and datasets, comprising 77 executable filtered notebooks, as shown in Figure 3(a).

Initial Task Construction. We first manually construct three seed tasks from representative filtered notebooks, guided by three principles: preserving the original analytical thread, formulating turns as quantitatively evaluable questions rather than visualization requests, and designing realistic long-range dependencies over evolving analytical states. We decompose each notebook into analysis clusters, i.e., groups of cells with shared computational objectives. Within each analysis cluster, we identify key analytical objects, such as data scopes, derived definitions, metrics, and intermediate results, and convert them into reusable analytical anchors. We then design state-dependent turns that reuse these anchors within and across analysis clusters to create long-range dependencies, following the **state-evolution patterns** in Section 3.1.

Based on the three manually constructed seed tasks, we use *Codex* (OpenAI, 2025), a coding agent equipped with *skill-creator*, to build a reusable *task-construction skill*. Specifically, we provide *Codex* with detailed design principles, three original notebooks, and corresponding converted seed tasks as paired demonstrations. The resulting skill encodes the construction procedure, including decomposition into analysis clusters, anchor identification, state-dependent turn design, and task annotations. We then use *Codex* with this *task-construction skill* to convert the remaining filtered notebooks into initial tasks. Each constructed task contains a long-horizon sequence of turns, where each turn includes a user request, executable reference code, a reference answer, state-evolution labels, and inter-turn dependency annotations. Together with the three seed tasks, this process yields 77 initial tasks, as shown in Figure 3(b).

Refinement and Validation. To ensure task quality, we conduct a three-stage refinement and validation process, as shown in Figure 3(c).

Stage 1: Expert Review. Expert reviewers with graduate-level training in data science and NLP manually inspect each task against three quality criteria: dependency validity, task difficulty, and answer reliability. The generated task annotations are first reviewed to verify that each turn depends on the correct prior states and that these dependencies are necessary for the intended analysis. Weakly dependent or overly simple turns are revised to require richer use of analytical states and longer-range dependencies. The reference code for the full task is then rerun to verify that it executes successfully and produces reproducible answers.

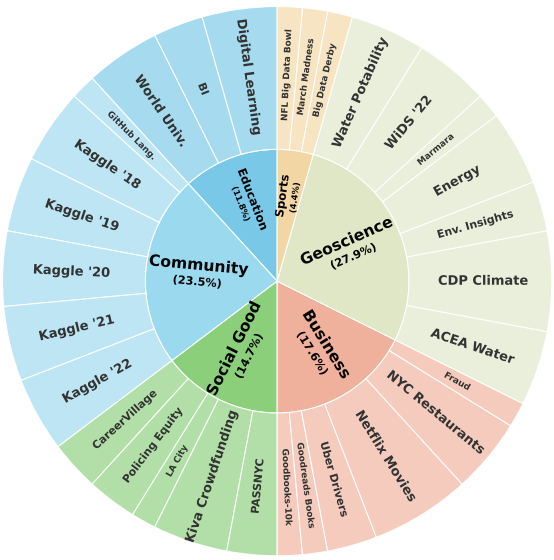


Figure 4: Domain and task distribution of LongDS. The inner ring shows application domains, while the outer ring shows source datasets and Kaggle competitions, with sector size proportional to the number of long-horizon analysis tasks.

Stage 2: Annotation-Guided Validation. We then use *Codex* for annotation-guided validation by providing all requests, data files, and task annotations for each reviewed task. This stage serves as an internal consistency check, using annotations to verify consistency among task specifications, dependencies, and reference answers. We compare *Codex* outputs with the reference answers and manually inspect mismatches to identify task-quality issues, such as ambiguous wording, missing information, incorrect annotations, or reference code errors. When such issues are found, we revise the task and rerun validation until no further task-quality issues are identified. Tasks whose ambiguity, weak dependency, or reliability issues cannot be resolved are discarded, resulting in 68 final tasks.

Stage 3: Final Task Check. Following the validation, we selectively remove redundant information from the final requests (e.g., restatements of earlier filters or metric definitions) so that long-range dependencies are not made overly explicit. After this removal step, we verify that each answer remains uniquely derivable from the final task specification and the provided data. Finally, we conduct a consistency check to ensure that the final requests, executable reference code, reference answers, and task annotations remain aligned.

3.3 Benchmark Statistics

Following the task curation pipeline and manual quality validation, LongDS contains **68** long-horizon data analysis tasks spanning six application domains: Sports, Geoscience, Business, Social Good, Education, and Community, as shown in Figure 4. Together, these tasks comprise **2,225** turns in total, with an average of 33 turns per task.

As shown in Appendix Figure 7, state-evolution patterns are frequent and diverse, with each task averaging 5.8 rollback turns and 8.6 multi-state composition turns, alongside frequent updates and counterfactual perturbations. Dependency structure is similarly demanding, with an average breadth of 2.9 dependencies per turn and an average span of 11.3 turns, confirming that LongDS requires agents to track and compose analytical states across long interaction histories. Detailed task-level macro statistics are provided in Appendix A.2.

We further examine potential training-data contamination arising from the use of public Kaggle notebooks. **Directed n-gram containment** indicates limited verbatim overlap with the source notebooks, while a **no-data memorization probe** yields near-zero performance without access to the underlying data. Full results are provided in Appendix C.4.

3.4 Evaluation Protocol

Each turn is paired with executable reference code and a structured reference answer, enabling reproducible evaluation. We use DeepSeek-V4-Pro (DeepSeek-AI, 2026) as the judge model (Liu et al., 2023; Zheng et al., 2023; Kim et al., 2024) to assess whether an agent's answer is semantically and numerically consistent with the reference answer, without constraining the output format. For turns involving model training, a small numerical tolerance is permitted to account for non-determinism.

Formally, the score $s_{i,j}$ for the j-th turn in the i-th task is defined as:

$$s_{i,j} = \begin{cases} 1, & \text{if consistent with the reference,} \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

For LongDS containing M tasks where the i-th task consists of N_i turns, the average score S_{avg} is defined as the macro-average of task-level scores:

$$S_{\text{avg}} = \frac{1}{M} \sum_{i=1}^{M} \left(\frac{1}{N_i} \sum_{j=1}^{N_i} s_{i,j} \right). \tag{2}$$

To validate the reliability of this automated evaluation protocol, we conduct a blind human audit, finding strong agreement between human and LLM judgments, with 93.11% agreement and Cohen's κ of 0.8623 (Bavaresco et al., 2025; Chiang and Lee, 2023; Landis and Koch, 1977). Further details are provided in Appendix D.2.

4 Experiments

4.1 Experiment Settings

We conduct our experiments using the DSGYM framework (Nie et al., 2026). The data analysis agent employs a ReAct-style strategy (Yao et al., 2023), generating reasoning traces and Python code executed in a persistent Jupyter Notebook kernel. Final answers are extracted from the agent's response to facilitate automated semantic evaluation.

We evaluate a diverse set of state-of-the-art LLMs on LongDS, including GPT-5.4 (OpenAI, 2026a), Gemini-3.1-Pro (Google Gemini Team, 2026), Claude-4.6-Sonnet (Anthropic, 2026), DeepSeek-V4-Pro, and Kimi-K2.6 (Moonshot AI, 2026). To accommodate complex multi-step reasoning and iterative debugging within long-horizon tasks, we set the maximum number of reasoning-action steps to 40 per turn. Further implementation details are provided in Appendix C.

4.2 Main Results

Overall Performance. Table 2 presents model results across the six domains in LongDS. Overall, even the best-performing model remains below 50% average accuracy, indicating that LongDS poses a substantial challenge for current LLM agents. **Gemini-3.1-Pro** achieves the highest average score, reaching **48.45**, and leads in Community, Social Good, Business, and Geoscience. GPT-5.4 and Claude-4.6-Sonnet follow with average scores of 43.50 and 41.56, respectively. Notably, **Kimi-K2.6** achieves an average score of 39.72, comparable to those of some proprietary models. We also evaluate *Codex* on a sampled subset of tasks, with details provided in Appendix C.2. On this subset, *Codex* improves over the ReAct-based Gemini-3.1-Pro baseline by 4.38 points, suggesting that a stronger agent does not fully resolve the long-horizon state-management challenge.

Domain Variance. Performance varies substantially across domains. Models generally achieve higher scores in Education, while lower performance is observed in Geoscience and Business, which often involve more complex feature engineering and long-horizon statistical reasoning. Bootstrap 95% confidence intervals further reveal greater uncertainty for smaller domains, as detailed in Appendix C.3. Overall, these cross-domain differences suggest that no single model consistently manages long-horizon analytical state across diverse domains.

Degradation in Long-Horizon State Tracking. Model performance degrades as long-horizon state tracking becomes more demanding. (1) *Accuracy decreases as tasks progress*. Figure 5(a) shows a nearly 47 percentage-point drop between the first and last 10% progress intervals after normalizing turn positions within each task. This decline suggests that agents struggle as analytical states accumulate. (2) *Dependency structure introduces an additional bottleneck*. Accuracy drops sharply as dependency breadth increases in Figure 5(b), with a similar decline under longer dependency spans in Figure 8. (3) *Performance declines as turns involve more complex state transitions.* Figure 5(c) shows a clear decline from Initial to Update, Counterfactual, and Rollback requests. This suggests that agents handle state construction relatively well, but struggle increasingly to revise, temporarily perturb, and restore analytical states. Together, these results show that LongDS stresses agents' ability to maintain, revise, restore, and compose analytical states across extended data-analysis trajectories.

5 Deep Analysis

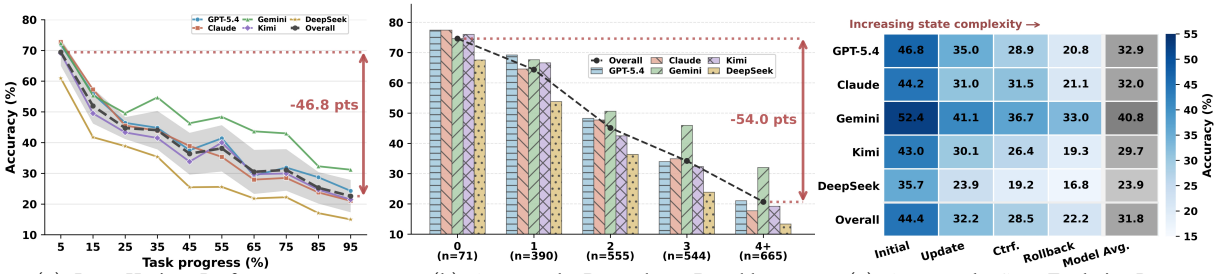
5.1 Efficiency and Performance Trade-off

As shown in Table 2, GPT-5.4 uses fewer agent steps than the other models, whereas Claude-4.6-Sonnet uses the most. Figure 6(a) further compares model performance with resource consumption, measured by agent steps and trajectory tokens. Gemini-3.1-Pro achieves the highest accuracy, while GPT-5.4 obtains the best cost-normalized efficiency due to its lower step and token usage. Claude-4.6-Sonnet uses the most steps but does not achieve the best accuracy, suggesting that more interaction budget does not necessarily improve long-horizon performance.

Domain-level results in Figure 6(b) show similar trends, with GPT-5.4 exhibiting strong resource efficiency across domains. Overall, these results suggest that **longer analysis is not inherently better** for long-horizon data analysis. A key factor is

Model	Education	Community	Social Good	Business	Geoscience	Sports	Avg Score	Avg Step
Proprietary Models								
GPT-5.4	**77.92**	65.32	36.80	28.40	28.90	10.52	43.50	68.57
Claude-4.6-Sonnet	77.29	54.64	36.10	25.54	31.92	19.76	41.56	170.04
Gemini-3.1-Pro	58.03	**69.54**	**41.73**	**33.59**	**42.20**	31.85	**48.45**	117.82
Open-source Models								
Kimi-K2.6	64.98	60.62	31.29	20.99	28.83	**32.85**	39.72	115.41
DeepSeek-V4-Pro	61.36	49.47	32.41	17.06	16.60	15.82	31.97	133.12

Table 2: **Main results across six domains in LongDS.** Scores are macro-averaged over task-level accuracies (%), with each task accuracy computed over its turns. Avg Step denotes average agent steps across all tasks. **Best** and second-best scores are highlighted, excluding Avg Step.



(a) Long-Horizon Performance (b) Accuracy by Dependency Breadth (c) Accuracy by State-Evolution Pattern

Figure 5: **Long-horizon performance degradation in LongDS.** Accuracy drops across three increasing demands: (a) later task progress, averaged within each 10% progress interval; (b) larger dependency breadth, with n denoting the number of turns per group; and (c) more complex state-evolution patterns.

whether the model can **maintain a correct analytical state** throughout the trajectory, as additional steps may introduce opportunities for state drift.

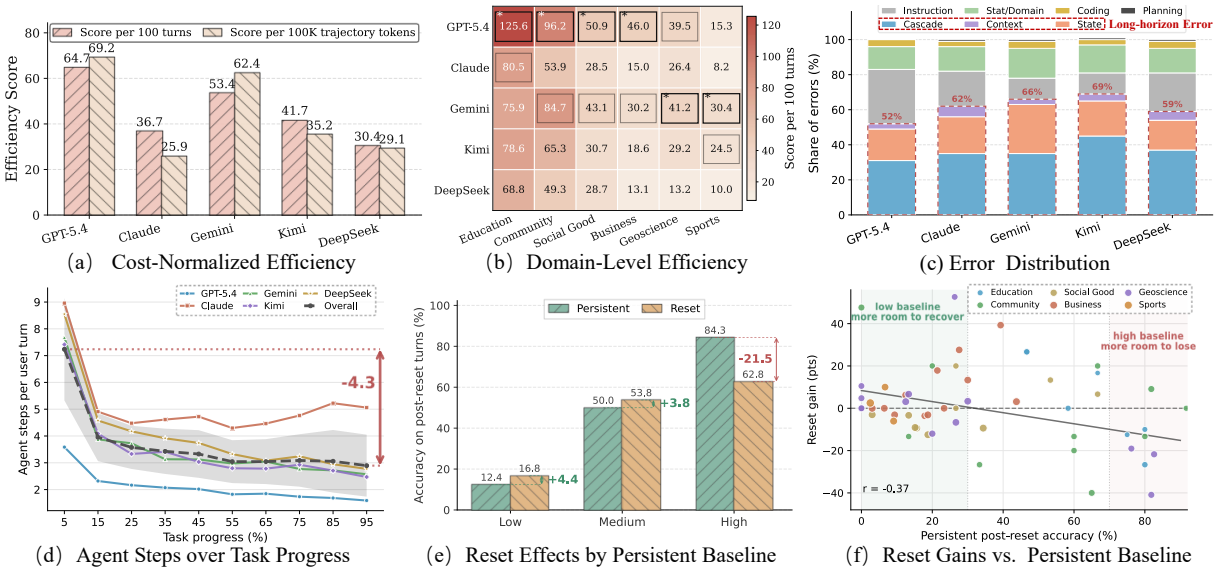
5.2 Error Analysis

Error Categorization. We first classify incorrect turns into general error types following DSGYM, including instruction following, statistical and domain reasoning, coding, and planning errors. To capture failures specific to long-horizon interactions, we additionally define three long-horizon error types: **Context Memory Error** refers to failures in recalling or using relevant historical information. **State Management Error** occurs when models select, update, or restore the wrong analytical state. **Cascade Error** denotes cases where the current turn is locally correct but fails due to incorrect intermediate states propagated from earlier turns. We use *Codex* to annotate the incorrect turns based on this taxonomy, and validate reliability through human auditing (Cohen's κ=0.75). Further details are provided in Appendix C.5.

Dominance of Long-Horizon Errors. Figure 6(c) illustrates the error distribution across models. Long-horizon errors account for the majority of failures, ranging from 52% for GPT-5.4 to 69% for Kimi-K2.6. This indicates that failures are driven more by long-horizon state reasoning than by coding, planning, or domain-reasoning errors. Among these errors, Cascade Error is the largest category, showing that incorrect intermediate states often propagate to later turns and affect downstream analyses. State Management Error also contributes substantially, reflecting failures in selecting, updating, or restoring the correct analytical state. By contrast, Context Memory Error occurs less frequently, suggesting that the main challenge is not merely retrieving prior information, but maintaining and applying the correct analytical state over time.

5.3 Persistent State and Reset Effects

Agentic Behavior Decreases over Long Trajectories. We analyze how models' exploration behavior evolves as tasks progress. As shown in Figure 6(d), the average number of agent steps per user turn decreases substantially, with the overall average dropping by **4.3** steps from early to late stages. This suggests that models explore the environment and establish code states in early turns, but increasingly rely on previously constructed states later. As a result, they perform less exploration, verification, and iterative refinement, which may make early state errors harder to detect and correct,



(a) Cost-Normalized Efficiency (b) Domain-Level Efficiency (c) Error Distribution

(d) Agent Steps over Task Progress (e) Reset Effects by Persistent Baseline (f) Reset Gains vs. Persistent Baseline

Figure 6: **Diagnosing the state-maintenance bottleneck in LongDS. (a) Cost-Normalized Efficiency:** Higher interaction cost does not necessarily yield better performance. **(b) Domain-Level Efficiency:** Efficiency varies across domains and models. **(c) Error Type Distribution:** Long-horizon errors, including Cascade, Context, and State errors, dominate failures. **(d) Agent Steps Over Task Progress:** Agents take fewer steps as tasks progress. **(e) Reset Effects by Persistent Baseline:** Reset helps weak persistent states but hurts strong ones. **(f) Reset Gains vs. Persistent Baseline:** Reset exposes a recovery–continuity trade-off.

and contribute to downstream cascading failures, as shown in Section 5.2.

Reset Experiment Setup. Motivated by the decrease in agent steps over long trajectories, we evaluate how resetting the code environment affects model performance. During task execution, we reset the environment once at a task-specific turn, as detailed in Appendix C.7. We then compute **accuracy only on turns after the reset** and compare it with the corresponding persistent baseline, where the same turns are evaluated without resetting the environment. To analyze how reset effects depend on the quality of the maintained state, we group tasks by their persistent accuracy on post-reset turns: Low (0–30%), Medium (30–70%), and High (70–100%).

Reset Trades Off Recovery and State Continuity. Intuitively, resetting the code environment would hurt performance, as it removes accumulated variables, intermediate results, and other useful execution state. However, Figure 6(e) shows a baseline-dependent effect: it slightly improves low- and medium-baseline cases, while substantially hurting high-baseline cases. This suggests that reset can help when the persistent code state has drifted, since it clears potentially erroneous execution state and requires the model to reconstruct the needed analytical state from the interaction history.

Figure 6(f) shows a negative correlation between persistent post-reset accuracy and reset gain. This pattern is consistent with reset helping degraded trajectories by reducing error propagation, while hurting strong trajectories by removing useful accumulated analytical state. Overall, code-environment reset trades off recovery from corrupted states against preservation of state continuity.

6 Related Work

Data Analysis Benchmarks and Agents. Existing data science benchmarks have progressed from isolated coding tasks to agentic and interactive data analysis settings. Code-oriented benchmarks mainly evaluate standalone data science programming problems (Lai et al., 2023; Huang et al., 2024b). Agentic benchmarks further require planning, code execution, tool use, and environment interaction within multi-step analytical workflows (Hu et al., 2024; Zhang et al., 2024; Gu et al., 2024; Jing et al., 2025; Zhang et al., 2025a; Egg et al., 2025; Chan et al., 2025; Wang et al., 2025c; Weng et al., 2025; Ma et al., 2026). Recent interactive benchmarks move closer to human data analysis by simulating analyst-agent collaboration and multi-round exploratory analysis (Dutta et al., 2025; Li et al., 2024a, 2025; Luo et al., 2026). In parallel, a growing line of data analysis agents automate end-to-end analytical pipelines via iterative reason-

ing and self-debugging (Rahman et al., 2025; Zhu et al., 2025; Hong et al., 2024; You et al., 2025; Qiao et al., 2025; Nie et al., 2026; Zhang et al., 2025b; Qiu et al., 2026). However, existing benchmarks and systems still emphasize task completion or workflow automation, leaving long-horizon analytical state management underexplored.

Long-Horizon and Multi-Turn Agent Evaluation. Recent work has studied LLM agents in multi-turn and long-horizon settings (Liu et al., 2025; Mialon et al., 2023), including dynamic user-agent interaction, tool use (Yao et al., 2024; Qin et al., 2023), web or API operations, and extended workflow completion (Zhou et al., 2024; Drouin et al., 2024; Xie et al., 2024). These benchmarks reveal performance degradation under sustained interaction, long-range consistency requirements, and changing task contexts (Luo et al., 2025b). However, they primarily target conversational tasks, web navigation, policy following, or general tool-use workflows rather than data analysis. LongDS differs by grounding long-horizon interaction in stateful data analysis environments, where agents must maintain, revise, restore, and compose evolving analytical states across extended analysis sessions. Extended discussion in Appendix B.

7 Conclusion

We introduce LongDS, a benchmark for evaluating long-horizon agentic data analysis in stateful data-analysis environments, where agents must maintain, update, restore, and compose evolving analytical states across extended interactions. Our results show that current proprietary and open-source models still struggle substantially in this setting, with performance degrading over longer trajectories and failures dominated by cascading and state-management errors. By making these long-horizon state-management limitations explicit, LongDS provides a challenging testbed for developing data-analysis agents that can more reliably manage analytical state over extended workflows.

Limitations

While LongDS provides a realistic benchmark for evaluating long-horizon agentic data analysis, several limitations remain.

First, LongDS is constructed from public Kaggle notebooks and datasets, which provide realistic analytical workflows but may not fully cover proprietary or production data-analysis scenarios. This also results in an imbalanced domain distribution, especially in Sports, where many candidate notebooks were filtered out due to large datasets or long-running computations.

Second, LongDS emphasizes quantitatively verifiable questions to support reliable evaluation, and therefore only partially covers open-ended insight generation, visualization-heavy analysis, and presentation-oriented analytics.

Third, LongDS uses a semi-automated task construction pipeline with Codex-assisted generation and expert-guided refinement. Although manual review helps ensure task quality, the resulting tasks may still reflect biases from the source notebooks or the construction process.

Ethics Statement

All datasets, notebooks, and the DSGYM framework used in this work are governed by their respective licenses, competition rules, and usage restrictions. We comply with all applicable terms. We do not collect any new personal data in constructing the benchmark, and our benchmark tasks do not involve private or sensitive personal information. Overall, we do not foresee any substantial ethical or societal concerns arising from this work.

Acknowledgements

We would like to express our sincere gratitude to the anonymous reviewers for their thoughtful and constructive feedback. This work was supported by the New Generation Artificial Intelligence-National Science and Technology Major Project (2025ZD0122802), National Natural Science Foundation of China (No. 62576307, No. NSFCU23B2055, No. NSFCU19B2027), the Fundamental Research Funds for the Central Universities (226-2023-00138), the Yongjiang Talent Introduction Programme (2021A-156-G), and the Information Technology Center and State Key Lab of CAD&CG, Zhejiang University. This work was also supported by Xinmiao Project of Zhejiang Provincial Applied Basic Research Program (2026XMZD064). This work was supported by Ant Group and Zhejiang University - Ant Group Joint Laboratory of Knowledge Graph.

References

Reyna Abhyankar, Qi Qi, and Yiying Zhang. 2026. Osworld-human: Benchmarking the efficiency of computer-use agents. *Preprint*, arXiv:2506.16042.

Anthropic. 2026. Claude sonnet 4.6. https://www.anthropic.com/news/claude-sonnet-4-6. Accessed: 2026-05-23.

antimo musone, Aredhel Bergström, Federico, Luisa Marotta, Maggie, and Maurizio Lucchesi. 2020. Acea smart water analytics. https://kaggle.com/competitions/acea-water-prediction. Kaggle.

Cody Bakley, Julia Elliott, Mary Styers, Scott McQuiggan, and Zarifa Zakaria. 2021. Learnplatform covid-19 impact on digital learning. https://kaggle.com/competitions/learnplatform-covid19-impact-on-digital-learning. Kaggle.

Anna Bavaresco, Raffaella Bernardi, Leonardo Bertolazzi, Desmond Elliott, Raquel Fernández, Albert Gatt, Esam Ghaleb, Mario Giulianelli, Michael Hanna, Alexander Koller, Andre Martins, Philipp Mondorf, Vera Neplenbroek, Sandro Pezzelle, Barbara Plank, David Schlangen, Alessandro Suglia, Aditya K Surikuchi, Ece Takmaz, and Alberto Testoni. 2025. LLMs instead of human judges? a large scale empirical study across 20 NLP evaluation tasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 238–255, Vienna, Austria. Association for Computational Linguistics.

BrendanGallegoBailey, Harsha Mallajosyula, igotcharts, Meghan O'Connell, Nicholas J. Matiasz, Paul Mooney, and Sari Ladin-Sienne. 2019. Data science for good: City of los angeles. https://kaggle.com/competitions/data-science-for-good-city-of-los-angeles. Kaggle.

Ruisheng Cao, Fangyu Lei, Haoyuan Wu, Jixuan Chen, Yeqiao Fu, Hongcheng Gao, Xinzhuang Xiong, Hanchong Zhang, Yuchen Mao, Wenjing Hu, Tianbao Xie, Hongshen Xu, Danyang Zhang, Sida Wang, Ruoxi Sun, Pengcheng Yin, Caiming Xiong, Ansong Ni, Qian Liu, Victor Zhong, Lu Chen, Kai Yu, and Tao Yu. 2024. Spider2-v: How far are multimodal agents from automating data science and engineering workflows? *Preprint*, arXiv:2407.10956.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. 2025. Mle-bench: Evaluating machine learning agents on machine learning engineering. *Preprint*, arXiv:2410.07095.

Ke Chen, Peiran Wang, Yaoning Yu, Xianyang Zhan, and Haohan Wang. 2025. Large language model-based data science agent: A survey. *Preprint*, arXiv:2508.02744.

Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. 2025. The browsergym ecosystem for web agent research. *Preprint*, arXiv:2412.05467.

Banghao Chi, Yining Xie, Mingyuan Wu, Jingcheng Yang, Jize Jiang, Zhaoheng Li, Shengyi Qian, Minjia Zhang, Klara Nahrstedt, Rui Hou, Xiangjun Fan, and Hanchao Yu. 2026. Spreadsheet-rl: Advancing large language model agents on realistic spreadsheet tasks via reinforcement learning. *Preprint*, arXiv:2605.22642.

Cheng-Han Chiang and Hung-yi Lee. 2023. Can large language models be an alternative to human evaluations? In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15607–15631, Toronto, Canada. Association for Computational Linguistics.

Chris Crawford and Jared Chung. 2019. Data science for good: Careervillage.org. https://kaggle.com/competitions/data-science-for-good-careervillage. Kaggle.

Camila Montes de Oca, Meghan O'Connell, Nicholas Clinton, Ollie Guinan, Paul Mooney, Saleem Van Groenou, and Simon Ilyushchenko. 2020. Ds4g - environmental insights explorer. https://kaggle.com/competitions/ds4g-environmental-insights-explorer. Kaggle.

DeepSeek-AI. 2026. Deepseek-v4: Towards highly efficient million-token context intelligence.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. 2024. Workarena: How capable are web agents at solving common knowledge work tasks? *Preprint*, arXiv:2403.07718.

Avik Dutta, Priyanshu Gupta, Hosein Hasanbeig, Rahul Pratap Singh, Harshit Nigam, Sumit Gulwani, Arjun Radhakrishna, Gustavo Soares, and Ashish Tiwari. 2025. Condabench: Interactive evaluation of language models for data analysis. *Preprint*, arXiv:2510.13835.

Adam DuVander, CDP Allison Hooks, Callum Richards, Katherine Walsh, Liz Posner, Meghan O'Connell, Paul Mooney, Reagan Swaine, SteliosTax, and Stephanie Lavallato. 2020. Cdp - unlocking climate solutions. https://kaggle.com/competitions/cdp-unlocking-climate-solutions. Kaggle.

Alex Egg, Martin Iglesias Goyanes, Friso Kingma, Andreu Mora, Leandro von Werra, and Thomas Wolf. 2025. Dabstep: Data agent benchmark for multi-step reasoning. *Preprint*, arXiv:2506.23719.

Julia Elliott and Paul Mooney. 2021. 2021 kaggle machine learning & data science survey. https://kaggle.com/competitions/kaggle-survey-2021. Kaggle.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Pal: Program-aided language models. *Preprint*, arXiv:2211.10435.

Google Gemini Team. 2026. Gemini 3.1 pro: A smarter model for your most complex tasks. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/. Accessed: 2026-05-23.

Ken Gu, Ruoxi Shang, Ruien Jiang, Keying Kuang, Richard-John Lin, Donghe Lyu, Yue Mao, Youran Pan, Teng Wu, Jiaqian Yu, Yikun Zhang, Tianmai M. Zhang, Lanyi Zhu, Mike A Merrill, Jeffrey Heer, and Tim Althoff. 2024. BLADE: Benchmarking language model agents for data-driven science. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 13936–13971, Miami, Florida, USA. Association for Computational Linguistics.

Siyuan Guo, Cheng Deng, Ying Wen, Hechang Chen, Yi Chang, and Jun Wang. 2024. Ds-agent: Automated data science by empowering large language models with case-based reasoning. *Preprint*, arXiv:2402.17453.

Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2025. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. *Preprint*, arXiv:2403.07714.

Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Chenxing Wei, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Xiangru Tang, Xiangtao Lu, Xiawu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Zhibin Gou, Zongze Xu, and Chenglin Wu. 2024. Data interpreter: An llm agent for data science. *Preprint*, arXiv:2402.18679.

Addison Howard, Ally Blake, Andrew Patton, Michael Lopez, Tom Bliss, and Will Cukierski. 2022a. Nfl big data bowl 2023. https://kaggle.com/competitions/nfl-big-data-bowl-2023. Kaggle.

Addison Howard, inversion, and Joe Appelbaum. 2022b. Big data derby 2022. https://kaggle.com/competitions/big-data-derby-2022. Kaggle.

Addison Howard and Jeff Sonas. 2020. Google cloud & ncaa® march madness analytics. https://kaggle.com/competitions/march-madness-analytics-2020. Kaggle.

Xueyu Hu, Ziyu Zhao, Shuang Wei, Ziwei Chai, Qianli Ma, Guoyin Wang, Xuwu Wang, Jing Su, Jingjing Xu, Ming Zhu, Yao Cheng, Jianbo Yuan, Jiwei Li, Kun Kuang, Yang Yang, Hongxia Yang, and Fei Wu. 2024. InfiAgent-DABench: Evaluating agents on data analysis tasks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 19544–19572. PMLR.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. 2024a. Mlagentbench: Evaluating language agents on machine learning experimentation. *Preprint*, arXiv:2310.03302.

Yiming Huang, Jianwen Luo, Yan Yu, Yitong Zhang, Fangyu Lei, Yifan Wei, Shizhu He, Lifu Huang, Xiao Liu, Jun Zhao, and Kang Liu. 2024b. Da-code: Agent data science code generation benchmark for large language models. *Preprint*, arXiv:2410.07331.

Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. 2026. Agentlab: Benchmarking llm agents against long-horizon attacks. *Preprint*, arXiv:2602.16901.

Liqiang Jing, Zhehui Huang, Xiaoyang Wang, Wenlin Yao, Wenhao Yu, Kaixin Ma, Hongming Zhang, Xinya Du, and Dong Yu. 2025. Dsbench: How far are data science agents from becoming data science experts? *Preprint*, arXiv:2409.07703.

Seungone Kim, Jamin Shin, Yejin Cho, Joel Jang, Shayne Longpre, Hwaran Lee, Sangdoo Yun, Seongjin Shin, Sungdong Kim, James Thorne, and Minjoon Seo. 2024. Prometheus: Inducing fine-grained evaluation capability in language models. *Preprint*, arXiv:2310.08491.

Yuhang Lai, Chengxi Li, Yiming Wang, Tianyi Zhang, Ruiqi Zhong, Luke Zettlemoyer, Wen-Tau Yih, Daniel Fried, Sida Wang, and Tao Yu. 2023. DS-1000: A natural and reliable benchmark for data science code generation. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 18319–18345. PMLR.

J Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics*, 33 1:159–74.

Hanyu Li, Haoyu Liu, Tingyu Zhu, Tianyu Guo, Zeyu Zheng, Xiaotie Deng, and Michael I. Jordan. 2025. Ida-bench: Evaluating llms on interactive guided data analysis. *Preprint*, arXiv:2505.18223.

Hongxin Li, Jingran Su, Yuntao Chen, Qing Li, and Zhaoxiang Zhang. 2023. Sheetcopilot: Bringing software productivity to the next level through large language models. *Preprint*, arXiv:2305.19308.

Huahang Li, Wentao Hu, Zhuoyue Wan, Chen Jason Zhang, Haoyang Li, and Xiaoyong Wei. 2026a. Dataclaw: An autonomous data agent with instant messaging integration. *Preprint*, arXiv:2604.24067.

Jinyang Li, Nan Huo, Yan Gao, Jiayi Shi, Yingxiu Zhao, Ge Qu, Yurong Wu, Chenhao Ma, Jian-Guang Lou, and Reynold Cheng. 2024a. Tapilot-crossing: Benchmarking and evolving llms towards interactive data analysis agents. *Preprint*, arXiv:2403.05307.

Yiyang Li, Zheyuan Zhang, Tianyi Ma, Zehong Wang, Keerthiram Murugesan, Chuxu Zhang, and Yanfang Ye. 2026b. Longda: Benchmarking llm

agents for long-document data analysis. *Preprint*, arXiv:2601.02598.

Ziming Li, Qianbo Zang, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, Wanjun Zhong, Wangchunshu Zhou, Wenhao Huang, and Ge Zhang. 2024b. Autokaggle: A multi-agent framework for autonomous data science competitions. *Preprint*, arXiv:2410.20424.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2025. Agentbench: Evaluating llms as agents. *Preprint*, arXiv:2308.03688.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-eval: NLG evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, Singapore. Association for Computational Linguistics.

An Luo, Jin Du, Xun Xian, Robert Specht, Fangqiao Tian, Ganghua Wang, Xuan Bi, Charles Fleming, Ashish Kundu, Jayanth Srinivasa, Mingyi Hong, Rui Zhang, Tianxi Li, Galin Jones, and Jie Ding. 2026. Agentds technical report: Benchmarking the future of human-ai collaboration in domain-specific data science. *Preprint*, arXiv:2603.19005.

An Luo, Xun Xian, Jin Du, Fangqiao Tian, Ganghua Wang, Ming Zhong, Shengchun Zhao, Xuan Bi, Zirui Liu, Jiawei Zhou, Jayanth Srinivasa, Ashish Kundu, Charles Fleming, Mingyi Hong, and Jie Ding. 2025a. AssistedDS: Benchmarking how external domain knowledge assists LLMs in automated data science. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 18017–18060, Suzhou, China. Association for Computational Linguistics.

Haotian Luo, Huaisong Zhang, Xuelin Zhang, Haoyu Wang, Zeyu Qin, Wenjie Lu, Guozheng Ma, Haiying He, Yingsha Xie, Qiyang Zhou, Zixuan Hu, Hongze Mi, Yibo Wang, Naiqiang Tan, Hong Chen, Yi R. Fung, Chun Yuan, and Li Shen. 2025b. Ultrahorizon: Benchmarking agent capabilities in ultra long-horizon scenarios. *Preprint*, arXiv:2509.21766.

Pingchuan Ma, Rui Ding, Shuai Wang, Shi Han, and Dongmei Zhang. 2023. InsightPilot: An LLM-empowered automated data exploration system. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 346–352, Singapore. Association for Computational Linguistics.

Ruiying Ma, Shreya Shankar, Ruiqi Chen, Yiming Lin, Sepanta Zeighami, Rajoshi Ghosh, Abhinav Gupta, Anushrut Gupta, Tanmai Gopal, and Aditya G. Parameswaran. 2026. Can ai agents answer your data questions? a benchmark for data agents. *Preprint*, arXiv:2603.20576.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. *Preprint*, arXiv:2303.17651.

Maggie, Valerie, and WiDS Datathon. 2022. Excellence in research award (phase ii). https://kaggle.com/competitions/phase-ii-widsdatathon2022. Kaggle.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. Gaia: a benchmark for general ai assistants. *Preprint*, arXiv:2311.12983.

Paul Mooney. 2019. 2019 kaggle machine learning & data science survey. https://kaggle.com/competitions/kaggle-survey-2019. Kaggle.

Paul Mooney. 2020. 2020 kaggle machine learning & data science survey. https://kaggle.com/competitions/kaggle-survey-2020. Kaggle.

Paul Mooney. 2022. 2022 kaggle machine learning & data science survey. https://kaggle.com/competitions/kaggle-survey-2022. Kaggle.

Moonshot AI. 2026. Kimi k2.6: Advancing open-source coding. https://www.kimi.com/blog/kimi-k2-6. Accessed: 2026-05-23.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Jinwoo Shin, Sercan Ö. Arık, and Tomas Pfister. 2025. Mle-star: Machine learning engineering agent via search and targeted refinement. *Preprint*, arXiv:2506.15692.

Jaehyun Nam, Jinsung Yoon, Jiefeng Chen, Raj Sinha, Jinwoo Shin, and Tomas Pfister. 2026. Ds-star: Data science agent for solving diverse tasks across heterogeneous formats and open-ended queries. *Preprint*, arXiv:2509.21825.

Fan Nie, Junlin Wang, Harper Hua, Federico Bianchi, Yongchan Kwon, Zhenting Qi, Owen Queen, Shang Zhu, and James Zou. 2026. Dsgym: A holistic framework for evaluating and training data science agents. *Preprint*, arXiv:2601.16344.

OpenAI. 2025. Openai codex. https://github.com/openai/codex. GitHub repository.

OpenAI. 2026a. Introducing gpt-5.4. https://openai.com/index/introducing-gpt-5-4/. Accessed: 2026-05-23.

OpenAI. 2026b. Introducing gpt-5.5. https://openai.com/index/introducing-gpt-5-5/. Accessed: 2026-05-23.

Yixin Ou, Yujie Luo, Jingsheng Zheng, Lanning Wei, Zhuoyun Yu, Shuofei Qiao, Jintian Zhang, Da Zheng, Yuren Mao, Yunjun Gao, Huajun Chen, and Ningyu Zhang. 2025. Automind: Adaptive knowledgeable agent for automated data science. *Preprint*, arXiv:2506.10974.

Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. Memgpt: Towards llms as operating systems. *Preprint*, arXiv:2310.08560.

Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, UIST '23, New York, NY, USA. Association for Computing Machinery.

Ruyi Qi, Zhou Liu, and Wentao Zhang. 2026. Datacross: A unified benchmark and agent framework for cross-modal heterogeneous data analysis. *Preprint*, arXiv:2601.21403.

Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, Minghua Ma, Pu Zhao, Si Qin, Xiaoting Qin, Chao Du, Yong Xu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2024. Taskweaver: A code-first agent framework. *Preprint*, arXiv:2311.17541.

Shuofei Qiao, Yanqiu Zhao, Zhisong Qiu, Xiaobin Wang, Jintian Zhang, Zhao Bin, Ningyu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, et al. 2025. Scaling generalist data-analytic agents. *arXiv preprint arXiv:2509.25084*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *Preprint*, arXiv:2307.16789.

Zhisong Qiu, Shuofei Qiao, Kewei Xu, Yuqi Zhu, Lun Du, Ningyu Zhang, and Huajun Chen. 2026. Rewarding the scientific process: Process-level reward modeling for agentic data analysis. *Preprint*, arXiv:2604.24198.

Mizanur Rahman, Amran Bhuiyan, Mohammed Saidul Islam, Md Tahmid Rahman Laskar, Ridwan Mahbub, Ahmed Masry, Shafiq Joty, and Enamul Hoque. 2025. Llm-based data science agents: A survey of capabilities, challenges, and future directions. *Preprint*, arXiv:2510.04023.

Zachary T. Rewolinski, Austin V. Zane, Hao Huang, Chandan Singh, Chenglong Wang, Jianfeng Gao, and Bin Yu. 2026. Sanity checks for agentic data science. *Preprint*, arXiv:2604.11003.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Preprint*, arXiv:2302.04761.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Preprint*, arXiv:2303.11366.

Josefa Lia Stoisser, Marc Boubnovski Martell, Sidsel Boldsen, Kaspar Märtens, and Robert Kitchen. 2026. Ambig-ds: A benchmark for task-framing ambiguity in data-science agents. *Preprint*, arXiv:2605.09698.

Ji Sun, Guoliang Li, Peiyao Zhou, Yihui Ma, Jingzhe Xu, and Yuan Li. 2025a. Agenticdata: An agentic data analytics system for heterogeneous data. *Preprint*, arXiv:2508.05002.

Maojun Sun, Ruijian Han, Binyan Jiang, Houduo Qi, Defeng Sun, Yancheng Yuan, and Jian Huang. 2025b. A survey on large language model-based agents for statistics and data science. *The American Statistician*, page 1–14.

Xiaolei Sun. 2026. Chimera: A multi-agent graph-temporal framework for interpretable medical insurance fraud detection. In *Proceedings of the 2026 International Conference on Human-Computer Interaction, Neural Networks and Deep Learning*, HNNDL '26, pages 79–85, New York, NY, USA. Association for Computing Machinery.

Ansh Tanwar. 2023. Global data on sustainable energy (2000-2020).

Zeeshan ul-hassan Usmani. 2017. My uber drives.

Chenglong Wang, Bongshin Lee, Steven Drucker, Dan Marshall, and Jianfeng Gao. 2025a. Data formulator 2: Iterative creation of data visualizations, with ai transforming data along the way. *Preprint*, arXiv:2408.16119.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. Voyager: An open-ended embodied agent with large language models. *Preprint*, arXiv:2305.16291.

Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. 2025b. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows. *Preprint*, arXiv:2508.09124.

Ziting Wang, Shize Zhang, Haitao Yuan, Jinwei Zhu, Shifu Li, Wei Dong, and Gao Cong. 2025c. Fdabench: A benchmark for data agents on analytical queries over heterogeneous data. *Preprint*, arXiv:2509.02473.

Han Weng, Zhou Liu, Yuanfeng Song, Xiaoming Yin, Xing Chen, and Wentao Zhang. 2025. Unidatabench: Evaluating data analytics agents across structured and unstructured data. *Preprint*, arXiv:2511.01625.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. 2023. Autogen: Enabling next-gen llm applications via multi-agent conversation. *Preprint*, arXiv:2308.08155.

Xianjie Wu, Jian Yang, Linzheng Chai, Ge Zhang, Jiaheng Liu, Xeron Du, Di Liang, Daixin Shu, Xianfu Cheng, Tianzhen Sun, et al. 2025. Tablebench: A comprehensive and complex benchmark for table question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25497–25506.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. 2024. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Preprint*, arXiv:2404.07972.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, Kritanjali Jain, Mengxue Bao, Zora Z. Wang, Xuhui Zhou, Zhitong Guo, Murong Cao, Mingyang Yang, Hao Yang Lu, Amaad Martin, Zhe Su, Leander Maben, Raj Mehta, Wayne Chi, Lawrence Jang, Yiqing Xie, Shuyan Zhou, and Graham Neubig. 2025. Theagentcompany: Benchmarking llm agents on consequential real world tasks. *Preprint*, arXiv:2412.14161.

Xu Yang, Xiao Yang, Shikai Fang, Yifei Zhang, Jian Wang, Bowen Xian, Qizheng Li, Jingyuan Li, Minrui Xu, Yuante Li, Haoran Pan, Yuge Zhang, Weiqing Liu, Yelong Shen, Weizhu Chen, and Jiang Bian. 2025. R&d-agent: An llm-agent framework towards autonomous data science. *Preprint*, arXiv:2505.14738.

Yibo Yang, Fei Lei, Yixuan Sun, Yantao Zeng, Chengguang Lv, Jiancao Hong, Jiaojiao Tian, Tianyu Qiu, Xin Wang, Yanbing Chen, Yanjie Li, Zheng Pan, Xiaochen Zhou, Guanzhou Chen, Haoran Lv, Yuning Xu, Yue Ou, Haodong Liu, Shiqi He, Anya Jia, Yulei Xin, Huan Wu, Liang Liu, Jiaye Ge, Jianxin Dong, Dahua Lin, and Wenxiu Sun. 2026. Aidabench: Ai data analytics benchmark. *Preprint*, arXiv:2603.15636.

Zhiyu Yang, Zihan Zhou, Shuo Wang, Xin Cong, Xu Han, Yukun Yan, Zhenghao Liu, Zhixing Tan, Pengyuan Liu, Dong Yu, Zhiyuan Liu, Xiaodong Shi, and Maosong Sun. 2024. Matplotagent: Method and evaluation for llm-based agentic scientific data visualization. *Preprint*, arXiv:2402.11453.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *Preprint*, arXiv:2406.12045.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. *Preprint*, arXiv:2210.03629.

Ziming You, Yumiao Zhang, Dexuan Xu, Yiwei Lou, Yandong Yan, Wei Wang, Huamin Zhang, and Yu Huang. 2025. DatawiseAgent: A notebook-centric LLM agent framework for adaptive and robust data science automation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 1099–1123, Suzhou, China. Association for Computational Linguistics.

Dan Zhang, Sining Zhoubian, Min Cai, Fengzu Li, Lekang Yang, Wei Wang, Tianjiao Dong, Ziniu Hu, Jie Tang, and Yisong Yue. 2025a. Datascibench: An llm agent benchmark for data science. *Preprint*, arXiv:2502.13897.

Kangning Zhang, Shuai Shao, Qingyao Li, Jianghao Lin, Lingyue Fu, Shijian Wang, Wenxiang Jiao, Yuan Lu, Weiwen Liu, Weinan Zhang, and Yong Yu. 2026. Mmskills: Towards multimodal skills for general visual agents. *Preprint*, arXiv:2605.13527.

Shaolei Zhang, Ju Fan, Meihao Fan, Guoliang Li, and Xiaoyong Du. 2025b. Deepanalyze: Agentic large language models for autonomous data science. *Preprint*, arXiv:2510.16872.

Wenqi Zhang, Yongliang Shen, Zeqi Tan, Guiyang Hou, Weiming Lu, and Yueting Zhuang. 2025c. Datacopilot: Bridging billions of data and humans with autonomous workflow. *Preprint*, arXiv:2306.07209.

Yuge Zhang, Qiyang Jiang, XingyuHan XingyuHan, Nan Chen, Yuqing Yang, and Kan Ren. 2024. Benchmarking data science agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5677–5700, Bangkok, Thailand. Association for Computational Linguistics.

Jingsheng Zheng, Jintian Zhang, Yujie Luo, Yuren Mao, Yunjun Gao, Lun Du, Huajun Chen, and Ningyu Zhang. 2026. Can we predict before executing machine learning agents? *Preprint*, arXiv:2601.05930.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *Preprint*, arXiv:2306.05685.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2023. Memorybank: Enhancing large language models with long-term memory. *Preprint*, arXiv:2305.10250.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. Webarena: A realistic web environment for building autonomous agents. *Preprint*, arXiv:2307.13854.

Wei Zhou, Jun Zhou, Haoyu Wang, Zhenghao Li, Qikang He, Shaokun Han, Guoliang Li, Xuanhe Zhou, Yeye He, Chunwei Liu, Zirui Tang, Bin Wang, Shen Tang, Kai Zuo, Yuyu Luo, Zhenzhe Zheng, Conghui He, Jingren Zhou, and Fan Wu. 2026. Can llms clean up your mess? a survey of application-ready data preparation with llms. *Preprint*, arXiv:2601.17058.

Yizhang Zhu, Liangwei Wang, Chenyu Yang, Xiaotian Lin, Boyan Li, Wei Zhou, Xinyu Liu, Zhangyang Peng, Tianqi Luo, Yu Li, Chengliang Chai, Chong Chen, Shimin Di, Ju Fan, Ji Sun, Nan Tang, Fugee Tsung, Jiannan Wang, Chenglin Wu, Yanwei Xu, Shaolei Zhang, Yong Zhang, Xuanhe Zhou, Guoliang Li, and Yuyu Luo. 2026. A survey of data agents: Emerging paradigm or overstated hype? *Preprint*, arXiv:2510.23587.

Yuqi Zhu, Yi Zhong, Jintian Zhang, Ziheng Zhang, Shuofei Qiao, Yujie Luo, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. 2025. Why do open-source llms struggle with data analysis? a systematic empirical study. *arXiv preprint arXiv:2506.19794*.

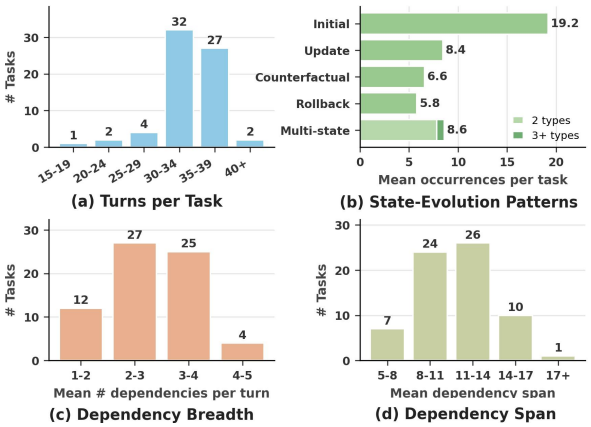


Figure 7: Benchmark statistics of LongDS. **(a) Turn Distribution**: Number of turns per task. **(b) State-Evolution Patterns**: Mean occurrences per task; Multi-state denotes turns annotated with two or more state types. **(c) Dependency Breadth**: Distribution of task-level mean number of direct prior-turn dependencies per turn. **(d) Dependency Span**: Distribution of task-level mean farthest prior-turn dependency distance, in number of turns.

A Details of Benchmark

A.1 State-Evolution Patterns

Table 3 summarizes the State-evolution patterns in LongDS. State inheritance is included for completeness, but it is treated as the default continuity assumption rather than a separate annotated category in benchmark statistics.

A.2 Additional Benchmark Statistics

Figure 7 shows the state-evolution patterns of LongDS. Table 4 reports the task-level macro statistics of LongDS.

A.3 Task sources

Table 14 lists the Kaggle competitions, public datasets, and analysis notebooks used to construct LongDS. These sources were selected to cover diverse real-world analytical domains and to provide executable workflows with sufficient analytical depth for long-horizon task construction. For each source, we report the corresponding task identifier, notebook title, and URL to make the benchmark provenance transparent and reproducible.

B Extended Related Work

Surveys on data science agents. Recent surveys organize LLM-based data science agents by capability, autonomy, data-science lifecycle stage, and application setting (Rahman et al., 2025; Zhu et al., 2026; Chen et al., 2025; Sun et al., 2025b). Work on LLM-based data preparation further covers data cleaning, transformation, and application-ready preparation pipelines (Zhou et al., 2026).

Data analysis and data science benchmarks. Prior benchmarks evaluate data-science coding, analytical reasoning, machine-learning experimentation, and agentic data workflows (Lai et al., 2023; Huang et al., 2024b; Hu et al., 2024; Zhang et al., 2024; Gu et al., 2024; Jing et al., 2025; Zhang et al., 2025a; Egg et al., 2025; Huang et al., 2024a; Chan et al., 2025; Luo et al., 2025a; Wang et al., 2025c; Weng et al., 2025; Ma et al., 2026). Other recent benchmarks broaden evaluation to long-document analysis, heterogeneous document and table settings, spreadsheet environments, task-framing ambiguity, multimodal data science workflows, and cross-modal analysis (Li et al., 2026b,a; Stoisser et al., 2026; Chi et al., 2026; Wu et al., 2025; Yang et al., 2026; Qi et al., 2026; Cao et al., 2024).

Interactive data analysis benchmarks. Interactive and conversational benchmarks evaluate agents under multi-round analytical interaction, guided analysis, or analyst-agent collaboration (Dutta et al., 2025; Li et al., 2024a, 2025; Luo et al., 2026).

Comparison with Existing Benchmarks. Table 1 compares LongDS with recent benchmarks for agentic and interactive data analysis. DABSTEP and LONGDA primarily evaluate single-turn data analysis tasks, while AGENTDS focuses on comparing AI-only performance with human–AI collaborative performance. Although CONDABENCH and IDA-BENCH support multi-turn interaction, their turns primarily serve to clarify underspecified requests or progressively provide task guidance.

In contrast, LongDS evaluates the long-horizon management of evolving analytical states across multi-turn data analysis workflows. Each turn constitutes a concrete analytical request with its own reference answer and turn-level evaluation. This design enables direct assessment of whether an agent can correctly inherit, update, counterfactually modify, restore, and compose analytical states over extended interactions.

Data analysis agents and systems. LLM-based data analysis systems automate exploration, spreadsheet manipulation, visualization, notebook-centered analysis, data-science pipelines, and

Pattern	Definition	Example
Initial	Establishes a **reusable analytical object**, such as a cohort, metric, rule, or intermediate result.	Define **high-activity users** as those with at least 10 sessions.
Inheritance	Reuses the **most recent valid analytical state** without restating it.	Using the **same user group**, compare retention across regions.
Update	Revises a previous definition, formula, filter, aggregation rule, or baseline, making the revision the **new default state**.	Use 20 sessions as the **new cutoff** for high-activity users in the **following analysis**.
Counterfactual	Introduces a **temporary alternative assumption** for the **current turn only**.	Recompute the result assuming a **5-session** cutoff instead.
Rollback	Answers under an **earlier anchored version** of the analysis instead of the most recent state.	Revisit the **initial high-activity** definition and recompute the result.
Composition	Combines **two or more explicit state operations** beyond default inheritance.	Use the **initial user group**, but evaluate it with the **revised retention metric**.

Table 3: State-evolution patterns in LongDS. Short labels denote initial state construction, state inheritance, state update, counterfactual perturbation, rollback, and multi-state composition, respectively. **Blue** highlights mark the key state semantics in each definition and example. Examples are illustrative.

Domain	Edu.	Comm.	Soc.	Bus.	Geo.	Spo.
# Tasks	8	16	10	12	19	3
Overall statistics	# Tasks = 68, # Domains = 6					
Turns / task						32.7
Initial / task						19.2
Update / task						8.4
Counterfactual / task						6.6
Rollback / task						5.8
Multi-state / task						8.6
Dependency breadth / turn						2.85
Dependency span / turn						11.29

Table 4: Benchmark scale and task-level macro statistics in LongDS. Mean values are first computed within each task and then averaged across all 68 tasks.

competition-style workflows (Ma et al., 2023; Li et al., 2023; Yang et al., 2024; Guo et al., 2024; Hong et al., 2024; You et al., 2025; Li et al., 2024b; Zhang et al., 2025c; Wang et al., 2025a). Recent systems and methods further emphasize scalable training, adaptive planning, verification, process supervision, and heterogeneous data analytics (Zhu et al., 2025; Qiao et al., 2025; Qiu et al., 2026; Rewolinski et al., 2026; Zheng et al., 2026; Nie et al., 2026; Zhang et al., 2025b; Ou et al., 2025; Yang et al., 2025; Nam et al., 2025, 2026; Sun et al., 2025a).

Long-horizon and tool-use agent evaluation. General agent benchmarks study multi-turn interaction, tool use, web navigation, office workflows, API use, and long-horizon task completion (Liu et al., 2025; Mialon et al., 2023; Yao et al., 2024; Qin et al., 2023; Guo et al., 2025; Zhou et al., 2024; Chezelles et al., 2025; Drouin et al., 2024; Xie et al., 2024; Wang et al., 2025b; Xu et al., 2025; Luo et al., 2025b; Abhyankar et al., 2026; Jiang et al., 2026; Zhang et al., 2026). Broader work on tool-using, reflective, memory-augmented, and multi-agent LLM systems provides additional context for agent design and evaluation (Schick et al., 2023; Gao et al., 2023; Wu et al., 2023; Qiao et al., 2024; Shinn et al., 2023; Madaan et al., 2023; Wang et al., 2023; Packer et al., 2024; Park et al., 2023; Sun, 2026; Zhong et al., 2023).

C Details of Experiment

C.1 Experimental Setup

The data analysis agent is instructed via a unified, structured system prompt designed to regulate its tool-use and reasoning format within the DSGYM framework. The system prompt is provided in Appendix E.1.

To establish a controlled evaluation environment, we standardize the generation parameters across all baseline models to the furthest extent permitted by their respective APIs. For GPT-5.4, Gemini-3.1-Pro, Claude-4.6-Sonnet, and DeepSeek-V4-Pro, the temperature is strictly set to 0.0. For Kimi-K2.6, the temperature is maintained at 1.0 because the official API implementation restricts temperature modification. Across all evaluated models, the maximum number of output tokens is capped at $8,192$ per interaction turn to accommodate extensive multi-step analytical outputs.

C.2 Codex Results on a Sampled Subset

Evaluating Codex requires a manual turn-by-turn interaction protocol: each task turn must be provided to Codex sequentially, and each turn-level answer must be manually collected before proceeding to the next turn. Due to this operational cost, we evaluate *Codex* (GPT-5.4, with high reasoning effort) on a domain-stratified sampled subset of LongDS, selecting two tasks from each of the six domains.

For comparison, Table 5 reports the performance of *Codex* and the other evaluated models on the same sampled subset. The results should therefore be interpreted as a complementary sampled-subset study rather than a replacement for the full benchmark results in Table 2. On this sampled subset, *Codex* achieves the highest average score of 65.55 and shows strong performance across several domains. These results suggest that a stronger code-centric agent can improve performance on some LongDS tasks, while long-horizon analytical state management remains challenging.

C.3 Statistical Information Details

To assess the statistical reliability of our results, we report bootstrap 95% confidence intervals for both **per-domain** and **per-pattern** performance, together with **three-run evaluations** of two representative models.

For the **domain-level results**, we perform task-level bootstrap resampling for each model–domain pair and recompute the average score for each bootstrap sample. This accounts for variability across tasks within each domain. The resulting confidence intervals are reported in Table 6. As expected, domains with fewer tasks exhibit wider confidence intervals.

For the **state-evolution pattern results**, pattern labels are assigned at the turn level, while multiple turns may originate from the same task and are therefore not independent. We consequently adopt a task-cluster bootstrap: tasks are resampled with replacement, and pattern-level accuracy is recomputed over all corresponding turns from the sampled tasks. This preserves within-task dependencies and avoids treating individual turns as independent observations. The resulting 95% confidence intervals are reported in Table 7.

For the **multi-run evaluation**, we conduct three independent runs for two representative models, GPT-5.4 and DeepSeek-V4-Pro, and report the mean and standard deviation across runs. As shown in Table 8, the overall scores exhibit relatively small run-to-run variation, with standard deviations of 0.81 for GPT-5.4 and 0.95 for DeepSeek-V4-Pro. Variation is larger for some individual domains, particularly those with fewer tasks.

C.4 Training-Data Contamination Analysis

Since LongDS is constructed from public Kaggle notebooks, we consider potential training-data contamination from two perspectives: mitigation during benchmark construction and empirical contamination probes.

Mitigation in Benchmark Construction. In LongDS, source notebooks serve only as references for the overall analytical workflow, while benchmark questions do not directly reuse the original analyses or answers. During task construction, we introduce new state-evolution operations, including updates, rollbacks, counterfactual perturbations, and compositions, yielding analytical trajectories that are absent from the source notebooks. In addition, LongDS requires structured, multi-field numerical answers, such as ranked lists of entities with their corresponding scores, rather than directly reusing notebook conclusions. These design choices reduce the extent to which benchmark tasks can be solved by recalling the original notebook trajectories or conclusions.

N-gram Containment. We assess potential verbatim overlap by comparing each complete benchmark task, including its questions, solution code, and answers, with the corresponding source notebook using directed n-gram containment. As shown in Table 9, containment decreases as n increases, from 2.32% for 4-grams to 0.94% for 13-grams. The low long-span containment rates suggest limited verbatim overlap between LongDS tasks and their source notebooks.

No-Data Memorization Probe. We further evaluate whether models can answer benchmark questions without access to the underlying data files. In this setting, models receive only the task context and question. As shown in Table 10, both GPT-5.4 and DeepSeek-V4-Pro achieve near-zero scores, with averages of 1.72 and 1.24, respectively. These results suggest that direct answer memorization is unlikely to account for the observed benchmark performance.

Model	Community	Education	Business	Geoscience	Social Good	Sports	Average
Codex	**56.67**	**86.67**	67.10	68.50	**80.00**	**34.34**	**65.55**
GPT-5.4	41.67	70.00	75.65	68.58	71.67	13.40	56.98
Claude-4.6	55.00	63.00	**77.97**	70.59	71.67	15.36	59.26
Gemini-3.1	55.00	75.00	70.95	**82.12**	73.34	15.63	61.17
Kimi-K2.6	41.67	73.00	62.61	77.56	73.34	19.51	57.78
DeepSeek-V4	38.34	69.34	66.45	19.04	76.67	11.84	46.61

Table 5: Codex results on a domain-stratified sampled subset of LongDS. We sample two tasks from each domain and evaluate Codex using a manual turn-by-turn protocol. All scores are accuracies (%) computed on the same sampled subset. **Best** and second-best scores are highlighted.

Model	Avg (n=68)	Edu (n=8)	Comm (n=16)	Soc (n=10)	Bus (n=12)	Geo (n=19)	Spo (n=3)
GPT	43.5 [36.1, 50.9]	77.9 [69.3, 86.3]	65.3 [51.5, 78.4]	36.8 [25.3, 50.5]	28.4 [22.3, 34.6]	28.9 [15.9, 43.3]	10.5 [4.8, 14.3]
Claude	41.6 [34.8, 48.4]	77.3 [63.8, 89.6]	54.6 [40.8, 67.9]	36.1 [22.8, 50.9]	25.5 [17.3, 33.8]	31.9 [20.5, 44.7]	19.8 [5.7, 28.6]
Gemini	48.5 [41.8, 55.0]	58.0 [39.4, 75.6]	69.5 [58.7, 78.5]	41.7 [28.0, 55.8]	33.6 [22.9, 43.9]	42.2 [29.3, 55.3]	31.9 [0.0, 64.3]
Kimi	39.7 [33.1, 46.3]	65.0 [44.3, 81.7]	60.6 [50.3, 70.1]	31.3 [16.8, 48.2]	21.0 [16.2, 26.1]	28.8 [17.6, 41.5]	32.9 [17.1, 59.5]
DeepSeek	32.0 [25.7, 38.4]	61.4 [42.1, 77.5]	49.5 [36.9, 61.8]	32.4 [19.3, 48.1]	17.1 [10.9, 25.2]	16.6 [9.5, 25.6]	15.8 [9.4, 23.8]

Table 6: Per-domain performance with 95% bootstrap confidence intervals. Edu, Comm, Soc, Bus, Geo, and Spo denote Education, Community, Social Good, Business, Geoscience, and Sports, respectively.

C.5 Error Analysis Details

Agent-as-Judge Annotation. We use *Codex* as an agent-as-judge to assist the annotation of incorrect turns with the prompt in Appendix E.3. Specifically, we run *Codex* with GPT-5.5 (OpenAI, 2026b) and x-high reasoning effort to examine the task context, reference answer, model response, and execution trace, and assign each incorrect turn to one of the predefined error categories. For each evaluated task, we provide *Codex* with four files: `code.py`, `ground_code.py`, `results_eval.json`, and `task.ipynb`. Together, these files contain the agent-generated code, reference solution code, turn-level questions and answers, evaluation outcomes, agent trajectories, and reference solution logic.

We sample six tasks from each domain, except for Sports where only three tasks are available, and annotate the results of all five evaluated models on each sampled task. In total, Codex produces error annotations for 3,207 incorrect turns, which form the annotated error pool used for subsequent analysis and human validation. The results are shown in Figure 6(c)

Human Validation of Error Annotations. We conduct two complementary human validation studies to assess the reliability of our error annotations.

First, in the **blind relabeling study**, we sample 200 error cases. Annotators are shown the task information and model outputs but not the original error labels, and are asked to independently assign the primary error type according to our taxonomy. For cases where annotators are uncertain, we conduct a follow-up discussion to clarify the applicable taxonomy criteria and finalize the annotation. This setting evaluates whether the taxonomy can be applied consistently without label hints.

Second, in the **reference verification study**, we sample another 200 error cases, aiming to cover different error categories as well as boundary cases that are difficult to distinguish. Annotators are shown the reference primary error type and its supporting evidence, and are asked to judge whether the reference label is appropriate; if they disagree, they provide a corrected primary error type.

In both studies, the 200 cases are approximately evenly assigned to three annotators, with each annotator reviewing about one third of the cases. Annotators are provided with the error taxonomy and annotation guidelines in Appendix E.3 before labeling. We compute agreement, Cohen's κ, and macro-F1 by comparing human annotations with the corresponding Codex-generated primary error labels.

As shown in Table 11, the blind relabeling study achieves 81.50% overall agreement with a Cohen's κ of 0.7535, indicating substantial agreement even when annotators do not see the original labels. The reference verification study yields higher agreement, with 89.00% agreement, a Cohen's κ of 0.8715, and a macro-F1 of 0.8898. These results

Model	Initial	Update	Counterfactual	Rollback
GPT	46.8 [38.8, 54.5]	35.0 [28.7, 41.9]	28.9 [21.3, 37.2]	20.8 [13.5, 29.0]
Claude	44.2 [37.5, 50.9]	31.0 [25.0, 37.4]	31.5 [24.6, 39.3]	21.1 [15.1, 27.8]
Gemini	52.4 [45.5, 58.8]	41.1 [34.0, 48.2]	36.7 [28.3, 45.3]	33.0 [24.3, 42.4]
Kimi	43.0 [36.2, 49.6]	30.1 [24.1, 37.0]	26.4 [20.0, 33.4]	19.3 [13.4, 25.9]
DeepSeek	35.7 [29.2, 42.1]	23.9 [18.7, 29.8]	19.2 [13.5, 25.7]	16.8 [10.7, 23.9]
Overall	44.4 [38.1, 50.6]	32.2 [27.0, 38.0]	28.5 [22.7, 34.9]	22.2 [16.5, 28.8]

Table 7: Performance across state-evolution patterns with 95% bootstrap confidence intervals.

Model	Edu	Comm	Soc	Bus	Geo	Spo	Avg
GPT-5.4	73.98 ± 4.96	59.07 ± 4.43	35.74 ± 3.09	31.01 ± 3.60	28.52 ± 1.54	24.38 ± 9.98	42.37 ± 0.81
DeepSeek-V4-Pro	64.09 ± 1.97	49.63 ± 1.29	29.95 ± 2.43	15.21 ± 1.51	22.26 ± 4.61	17.40 ± 5.42	33.29 ± 0.95

Table 8: Three-run evaluation results reported as mean ± standard deviation. Edu, Comm, Soc, Bus, Geo, and Spo denote Education, Community, Social Good, Business, Geoscience, and Sports, respectively.

Metric	Mean ± Std
4-gram containment	2.32 ± 4.14%
6-gram containment	1.63 ± 3.60%
8-gram containment	1.32 ± 3.28%
10-gram containment	1.13 ± 3.03%
13-gram containment	0.94 ± 2.73%

Table 9: Directed n-gram containment between LongDS tasks and their corresponding source notebooks.

Model	Avg. Score
DeepSeek-V4-Pro	1.24 ± 0.16
GPT-5.4	1.72 ± 0.16

Table 10: No-data memorization probe. Models are evaluated without access to the underlying data files. Results are reported as mean ± standard deviation.

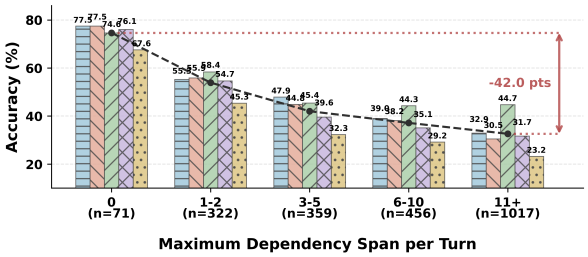


Figure 8: **Accuracy by dependency span.** Model accuracy decreases as the farthest dependency span becomes longer, indicating that agents struggle when the required analytical state must be recovered from more distant prior turns. n denotes the number of turns in each group.

suggest that the error taxonomy is generally reproducible under human annotation, while the lower macro-F1 in blind relabeling indicates that some boundary cases remain difficult to distinguish.

C.6 Dependency Span Analysis

Figure 8 complements the dependency-breadth analysis in the main text. While dependency breadth measures how many prior turns a request directly depends on, dependency span measures the maximum distance to the farthest depended prior turn. Accuracy declines as dependency span increases, showing that long-range analytical dependencies further challenge agents' ability to recover and apply the correct analytical state.

C.7 Reset Experiment

We conduct the reset experiment with GPT-5.4. We exclude five tasks for which the persistent baseline is either entirely incorrect or answers only one turn correctly, since such cases provide too little reliable state to analyze reset effects. For each remaining task, we run the agent in the standard persistent setting and then reset the executable environment once at a task-specific turn. The reset clears the accumulated code state, variables, and intermediate results in the execution environment, while the interaction history remains available to the agent.

To choose the reset point, we use a simple task-specific heuristic over four predefined candidate turns, 2, 4, 6, and 15. The candidate set covers early and middle stages of the trajectory while leaving enough turns for post-reset evaluation. For each task, we select the candidate whose remaining-turn ratio is closest to half of the task's persistent baseline accuracy. This heuristic is used only to define a consistent reset location for diagnostic comparison; all reported reset effects are computed on the same post-reset turns for both the reset run and the persistent baseline.

Study	Annotator	Cases	Agreement	Cohen's κ	Macro-F1
Blind relabeling	Human 1	67	82.09	0.7587	0.6403
Blind relabeling	Human 2	67	77.61	0.7094	0.5306
Blind relabeling	Human 3	66	84.85	0.7948	0.6725
Blind relabeling	Overall	200	81.50	0.7535	0.6177
Reference verification	Human 1	67	88.06	0.8605	0.8791
Reference verification	Human 2	67	94.03	0.9302	0.9402
Reference verification	Human 3	66	84.85	0.8228	0.8540
Reference verification	Overall	200	89.00	0.8715	0.8898

Table 11: Human validation results for error annotation reliability. Agreement is reported as percentage accuracy, while Cohen's κ and macro-F1 are computed against the original or reference primary error type.

Annotator	Cases	Agreement	Cohen's κ	Macro-F1
Human 1	150	92.00	0.8403	0.9199
Human 2	150	94.67	0.8933	0.9466
Human 3	150	92.67	0.8533	0.9266
Overall	450	93.11	0.8623	0.9311

Table 12: Human audit results for LLM-as-judge evaluation. Agreement is computed between blind human judgments and the original LLM-as-judge scores.

LLM-as-judge score	Human score	
	0	1
0	212	9
1	22	207

Table 13: Confusion matrix between blind human judgments and LLM-as-judge scores.

D Details of Evaluation

D.1 Evaluation Prompt

The complete prompt template utilized for the automated LLM-as-a-judge evaluation is presented in E.2.

D.2 Human Validation of the LLM Evaluator

To validate the reliability of the automated LLM-as-judge evaluation, we conduct a blind human audit on 450 evaluated samples. The sample is balanced across both domains and models, with 75 cases from each of the six domains and 90 cases from each of the five evaluated models. We also balance the original judge labels, including 225 cases judged correct and 225 cases judged incorrect by the automated evaluator.

The 450 cases are split into three groups of 150 cases, each reviewed by one of three annotators. Annotators are shown only the task question, ground-truth answer, and model response, without access to the original judge score or judge rationale. They are given the same evaluation instructions as the LLM-as-judge prompt in Appendix E.2 and independently make a binary decision on whether the model response correctly answers the question.

We then compare the human judgments with the original LLM-as-judge scores and report agreement, Cohen's κ, and macro-F1. As shown in Table 12, the blind human audit shows high agreement between human judgments and the LLM-as-judge scores. Overall agreement reaches 93.11%, with a Cohen's κ of 0.8623 and a macro-F1 of 0.9311. These results suggest that the automated evaluator is reliable for turn-level answer correctness evaluation.

Table 13 further shows the confusion matrix between blind human judgments and LLM-as-judge scores. Among 450 audited cases, 419 receive the same label from humans and the automated judge, while 31 cases differ. The automated judge marks 22 human-incorrect answers as correct and 9 human-correct answers as incorrect.

E Prompt Templates

E.1 System Prompt

System Prompt

```
You are an expert data scientist,
    statistical analyst and machine
    learning engineer who tackles
    analytical or machine learning
    challenges through systematic
    thinking and investigation.
For each task, you will receive a
    question along with file paths to
    the relevant data and background
    information in `{PATH}`.
Your goal is to:
1. Understand the problem - interpret
    the question, data format, and
    expected output format.
2. Explore and preprocess the data -
    load the datasets, perform data
```

Table 14: Kaggle Datasets and Notebooks in the Business Domain

Competition/Dataset	Task ID	Notebook Title	URL
goodbooks-10k	task1	Netflix Vs Books-Recommender, Analysis, EDA	https://www.kaggle.com/code/niharika41298/netflix-vs-books-recommender-analysis-eda
Goodreads-books	task1	Book Recommendation Engine	https://www.kaggle.com/code/snanilim/book-recommendation-engine
My Uber Drives (ul-hassan Usmani, 2017)	task1	UBER Rides Dataset 2016 ANALYSIS	https://www.kaggle.com/code/suiyue/uber-rides-dataset-2016-analysis
	task2	Uber_ride Analysis	https://www.kaggle.com/code/saurav9786/uber-ride-analysis
Netflix Movies and TV Shows	task1	Netflix Visualizations, Recommendation, EDA	https://www.kaggle.com/code/niharika41298/netflix-visualizations-recommendation-eda
	task2	storytelling with Data - Netflix ver.	https://www.kaggle.com/code/subinium/storytelling-with-data-netflix-ver
	task3	NETFLIX CONSUMPTION ANALYSIS	https://www.kaggle.com/code/sahib12/netflix-consumption-analysis/notebook
	task4	Netflix Data Visualization	https://www.kaggle.com/code/joshuaswords/netflix-data-visualization
NYC Restaurants Data - Food Ordering and Delivery	task1	NYC Restaurant Food Order & Delivery Detailed EDA	https://www.kaggle.com/code/ahsan81/nyc-restaurant-food-order-delivery-detailed-eda
	task2	Exploratory Data Analysis - NYC FoodHub	https://www.kaggle.com/code/lilyhyseni/exploratory-data-analysis-nyc-foodhub
	task3	Delivery Time - EDA, Grouping and ML (32%)	https://www.kaggle.com/code/raphaelmarconato/delivery-time-eda-grouping-and-ml-32
Transaction Data for fraud analysis	task1	Transaction Data for fraud analysis	https://www.kaggle.com/code/neamulislamfahim/transaction-data-for-fraud-analysis

Table 15: Kaggle Datasets and Notebooks in the Education Domain

Competition/Dataset	Task ID	Notebook Title	URL
BI intro to data cleaning eda and machine learning	task1	BI Data Cleaning, EDA and Predictive Modeling	https://www.kaggle.com/code/lukhilaksh/bi-data-cleaning-eda-and-predictive-modeling
	task2	notebook524401d43e	https://www.kaggle.com/code/walekhwatlphilip/notebook524401d43e
LearnPlatform COVID-19 Impact on Digital Learning (Bakley et al., 2021)	task1	Maslow Before Bloom	https://www.kaggle.com/code/iamleonie/maslow-before-bloom/input
	task2	Learning in Cyberspace: a Story of Pandemic Times	https://www.kaggle.com/code/mauromauro/learning-in-cyberspace-a-story-of-pandemic-times/notebook#8.-Wrap-up
	task3	Digital Learning During Pandemic-Contest Winner	https://www.kaggle.com/code/charliezimmerman/digital-learning-during-pandemic-contest-winner/notebook?scriptVersionId=87449728
World University Rankings	task1	World University Rankings Advanced Analysis	https://www.kaggle.com/code/gpreda/world-university-rankings-advanced-analysis#conclusions
	task2	Which universities do good science?	https://www.kaggle.com/code/pozdniakov/which-universities-do-good-science
	task3	MSU vs Top-7	https://www.kaggle.com/code/ospanoff/msu-vs-top-7/notebook

Table 16: Kaggle Datasets and Notebooks in the Geoscience Domain

Competition/Dataset	Task ID	Notebook Title	URL
Acea Smart Water Analytics (antimo musone et al., 2020)	task1	Intro to Time Series Forecasting	https://www.kaggle.com/code/iamleonie/intro-to-time-series-forecasting
	task2	How virtual trees can save real water in Italy?	https://www.kaggle.com/code/michau96/how-virtual-trees-can-save-real-water-in-italy
	task3	Acea Smart Water: Full EDA & Prediction	https://www.kaggle.com/code/maksymshkliarevskyi/acea-smart-water-full-eda-prediction
CDP - Unlocking Climate Solutions (DuVander et al., 2020)	task1	KPIs for measuring Climate Action and Inequality	https://www.kaggle.com/code/mannmann2/kpis-for-measuring-climate-action-and-inequality
	task2	CDP Challenge: Climate Adaptation Index	https://www.kaggle.com/code/shabou/cdp-challenge-climate-adaptation-index/report
	task3	CDP: A Path to Efficient and Sustainable Growth	https://www.kaggle.com/code/katemelianova/cdp-a-path-to-efficient-and-sustainable-growth
	task4	Impact Potential Analysis of Water-Use Efficiency	https://www.kaggle.com/code/iamleonie/impact-potential-analysis-of-water-use-efficiency
DS4G - Environmental Insights Explorer (de Oca et al., 2020)	task1	DS4G: An analytical approach to NO2 emissions	https://www.kaggle.com/code/chrisarderne/ds4g-an-analytical-approach-to-no2-emissions/notebook
	task2	DS4G: Spatial Panel Data Modeling	https://www.kaggle.com/code/katemelianova/ds4g-spatial-panel-data-modeling
Global Data on Sustainable Energy (2000-2020) (Tanwar, 2023)	task1	Starter Notebook: Global Sustainable Energy	https://www.kaggle.com/code/anshtanwar/starter-notebook-global-sustainable-energy
	task2	EcoOpt	https://www.kaggle.com/code/ahmadihossein/ecoopt
	task3	EDA |CO2 emission Data | Visualization	https://www.kaggle.com/code/abdallhaosman/eda-co2-emission-data-visualization
Marmara Region Earthquakes (Apr 23–24, 2025)	task1	Istanbul Quake Watch: Forecasting the Megaquake	https://www.kaggle.com/code/pinuto/istanbul-quake-watch-forecasting-the-megaquake
Excellence in Research Award (Phase II) (Maggie et al., 2022)	task1	WIDS II 2022: EDA	https://www.kaggle.com/code/sytuannguyen/wids-ii-2022-eda
	task2	Learning with Our Vulnerability: Covid-19	https://www.kaggle.com/code/mpwolke/learning-with-our-vulnerability-covid-19/notebook
	task3	How well (or not) we live - Health Rankings WiDS	https://www.kaggle.com/code/mpwolke/how-well-or-not-we-live-health-rankings-wids
Water Potability	task1	Water Potability Chemistry Instability Analysis	https://www.kaggle.com/datasets/adityakadiwal/water-potability
	task2	Water Potability Drinking-Status Modeling	https://www.kaggle.com/datasets/adityakadiwal/water-potability
	task3	Water Potability Safety Screening and Model Comparison	https://www.kaggle.com/datasets/adityakadiwal/water-potability

Table 17: Kaggle Datasets and Notebooks in the Social Good Domain

Competition/Dataset	Task ID	Notebook Title	URL
Data Science for Good: Center for Policing Equity	task1	Police Dogs and Grey hair will save you from jail	https://www.kaggle.com/code/harriken/police-dogs-and-grey-hair-will-save-you-from-jail/report
	task2	Very Detailed Analysis of CPE - DS for Good Winner	https://www.kaggle.com/code/ambarish/very-detailed-analysis-of-cpe-ds-for-good-winner/comments?scriptVersionId=8384365
Data Science for Good: City of Los Angeles (BrendanGallegoBailey et al., 2019)	task1	Phrasing: Improving Diversity Through Formatting	https://www.kaggle.com/code/filthyilliterate/phrasing-improving-diversity-through-formatting/notebook
Data Science for Good: Kiva Crowdfunding	task1	Kiva Data Exploration	https://www.kaggle.com/code/gpreda/kiva-data-exploration/report
	task2	Simple Exploration Notebook - Kiva	https://www.kaggle.com/code/sudalairajkumar/simple-exploration-notebook-kiva/notebook
	task3	ExtenKiva Exploration - EDA	https://www.kaggle.com/code/kabure/extenkiva-exploration-eda/notebook
CareerVillage.org (Crawford and Chung, 2019)	task1	Deepdive into careervillage	https://www.kaggle.com/code/infocusp/deepdive-into-careervillage
	task2	'When I grow up I want to be .. '	https://www.kaggle.com/code/arjundas/when-i-grow-up-i-want-to-be#Is-this-people-answers-specific-tags?
Data Science for Good: PASSNYC	task1	Target Schools & Action Recommended for PASSNYC	https://www.kaggle.com/code/laiyipeng/target-schools-action-recommended-for-passnyc?scriptVersionId=5753794
	task2	Recommendations to PASSNYC based on Data Analysis	https://www.kaggle.com/code/infocusp/recommendations-to-passnyc-based-on-data-analysis/notebook

Table 18: Kaggle Datasets and Notebooks in the Sports Domain

Competition/Dataset	Task ID	Notebook Title	URL
Big Data Derby 2022 (Howard et al., 2022b)	task1	Monitoring Racing Strategies for Injury Prevention	https://www.kaggle.com/code/iamleonie/monitoring-racing-strategies-for-injury-prevention/
Google Cloud & NCAA March Madness Analytics (Howard and Sonas, 2020)	task1	What Makes a Second-Half Team?	https://www.kaggle.com/code/hmtessier/what-makes-a-second-half-team/notebook
NFL Big Data Bowl 2023 (Howard et al., 2022a)	task1	NFL Data Bowl 2023: Initial Pass Set Kick Speed	https://www.kaggle.com/code/morganmartin23/nfl-data-bowl-2023-initial-pass-set-kick-speed

Table 19: Kaggle Datasets and Notebooks in the Community Domain

Competition/Dataset	Task ID	Notebook Title	URL
2018 Kaggle Machine Learning & Data Science Survey	task1	AfricAI	https://www.kaggle.com/code/mhajabri/africai
	task2	The MOOC Wars: Kaggle's Perspective	https://www.kaggle.com/code/ogakulov/the-mooc-wars-kaggle-s-perspective?scriptVersionId=8041710
	task3	Measuring Accountability in DS and ML with Waffles	https://www.kaggle.com/code/strangemane/measuring-accountability-in-ds-and-ml-with-waffles
2019 Kaggle Machine Learning & Data Science Survey (Mooney, 2019)	task1	Exploring PhD Community with Network Analysis	https://www.kaggle.com/code/artvolgin/exploring-phd-community-with-network-analysis
	task2	Is there any job out there? Kaggle vs Glassdoor	https://www.kaggle.com/code/andresionek/is-there-any-job-out-there-kaggle-vs-glassdoor/notebook
	task3	Spending $$$ for MS in Data Science - Worth it ?	https://www.kaggle.com/code/shivamb/spending-for-ms-in-data-science-worth-it
2020 Kaggle Machine Learning & Data Science Survey (Mooney, 2020)	task1	Treasure Hunt - what gives to be REALLY good?	https://www.kaggle.com/code/andradaolteanu/treasure-hunt-what-gives-to-be-really-good
	task2	Tools of the Trade: A Short History	https://www.kaggle.com/code/haakakak/tools-of-the-trade-a-short-history/notebook
	task3	How to make money in 2021	https://www.kaggle.com/code/viveknest/how-to-make-money-in-2021
2021 Kaggle Machine Learning & Data Science Survey (Elliott and Mooney, 2021)	task1	Data Science in 2021 : Adaptation or Adoption?	https://www.kaggle.com/code/shivamb/data-science-in-2021-adaptation-or-adoption
	task2	How are the Ladies and the Gents doing?	https://www.kaggle.com/code/andradaolteanu/how-are-the-ladies-and-the-gents-doing
	task3	Data Scientists & Analysts: What's the difference?	https://www.kaggle.com/code/spitfire2nd/data-scientists-analysts-what-s-the-difference/notebook
2022 Kaggle Machine Learning & Data Science Survey (Mooney, 2022)	task1	15 factors for data science in your country!	https://www.kaggle.com/code/michau96/15-factors-for-data-science-in-your-country
	task2	The State of Low / No-code in Data	https://www.kaggle.com/code/spitfire2nd/the-state-of-low-no-code-in-data/
	task3	Classifying Users and Learning From Experts	https://www.kaggle.com/code/rosspmcdonald/classifying-users-and-learning-from-experts/notebook
GitHub Programming Languages Data	task1	Data Visualization on Github Languages Data	https://www.kaggle.com/code/varunnagpalspyz/data-visualization-on-github-languages-data

```
    cleaning, feature engineering, and
    exploratory analysis where helpful.
3. Decompose the question and perform
    planning - break down the question
    into smaller steps and perform each
    step systematically. Change your
    plan if needed.
4. Analyze the data - build appropriate
    statistical models, causal models,
    machine learning models, or other
    analyses to answer the research
    question.
5. Generate final answer - provide a
    clear, specific answer to the
    question based on your analysis and
    the requirements.
6. Explain reasoning - clearly
    communicate assumptions,
    methodology, and trade-offs at each
    step.

TASK: Tackle the given data science
    question by analyzing the provided
    data to generate a final answer.

Important rules:
- Do not use plotting libraries (assume
    you cannot view plots). Use
    text-based summaries and statistics
    instead.
- Your final answer should be specific
    and directly address the question.
- For numerical answers, provide the
    exact value requested (rounded as
    specified if mentioned).
- Only produce the final answer when you
    have enough evidence and validation
    to support your approach.
- Try different approaches or perform
    deeper reasoning when you are
    uncertain about the answer.
- Code execution is continuous -
    variables and data loaded in
    previous steps remain available for
    subsequent analysis. Do not need to
    reload the same dataset or variables.
- Your code can only do one step at a
    time even when multiple steps are
    planned. Perform the next step based
    on the previous step's results.
- When calculation is needed, you are
    encouraged to use python code
    instead of calculating by yourself.
- You must provide your final answer in
    the format: <answer>your final
    answer</answer>

You MUST use the following format for
    your response. Each step must follow
    this exact structure:
<reasoning>
Write clear reasoning about what you
    plan to do next and why. Be specific
    about your analytical approach.
</reasoning>
<python>
Write executable Python code here. Each
    code block should do ONE specific
    task.Code must be complete and
    runnable. Include all necessary
    imports.
</python>
<information>
The output/results from your Python code
    will appear here.\nThis section is
    read-only - you cannot write here.
</information>
Repeat these blocks for each analysis
    step. When you reach your
    conclusion, you should follow this
    structure:
<reasoning>
Write clear reasoning about how you came
    up with your final answer.
</reasoning>
<answer>
Write your final answer here according
    to the requirements of the question.
    Do not include any other text or
    unnecessary information.
</answer>
```

E.2 LLM-as-Judge Prompt

Eval Prompt

```
## Evaluation Task

You are a strict factual evaluator. Your
    job is to check whether an agent's
    solution correctly answers the
    question by verifying it against the
    relevant facts in
the ground truth.

---

### Inputs

**Question:**
{question}

**Ground Truth (JSON):**
{ground_truth}

**Agent's Solution:**
{solution}

---

### Evaluation Rules

1. **Question-Driven Coverage** - First,
    analyze the `question` to determine
    which specific information is
    requested. You ONLY need to evaluate
    the fields in the
`ground_truth` that directly answer the
    question. Ignore extra fields in the
    `ground_truth` that are not
    requested. Ignore extra information
    in the solution as
well, as long as all required
    information is present and correct.
```

```
    Missing required fields count as
    incorrect.

2. **Numeric values** - Numeric answers
    must match the ground truth exactly
    after ignoring insignificant
    trailing zeros.

- Compare numeric values exactly after
    normalizing trailing zeros after the
    decimal point.
- Trailing zeros after the decimal point
    are insignificant and should be
    ignored.
- A decimal point followed only by zeros
    is equivalent to an integer.
- Do NOT round values.
- Do NOT allow +/- 1 tolerance in the
    last digit.
- Do NOT compare using fewer decimal
    places unless the removed digits are
    only trailing zeros.
- Percent signs, currency symbols,
    commas, and surrounding text may be
    ignored for parsing, but the numeric
    value itself must still match
    exactly after trailing-zero
    normalization.

Examples:
- Ground Truth `22245.00` vs Solution
    `22245` -> Match
- Ground Truth `25.7600` vs Solution
    `25.76` -> Match
- Ground Truth `0.125` vs Solution
    `0.12` -> Wrong, numeric value
    differs

If the ground truth explicitly includes
    a `tolerance` or `tolerance_note`
    field for a required numeric value,
    apply that tolerance only to the
    numeric value. Trailing zeros may
    still be ignored unless the
    tolerance note explicitly requires
    fixed formatting.

3. **Numeric tolerance** - If the ground
    truth explicitly includes a
    `tolerance` or `tolerance_note`
    field for a required numeric value:
- Apply that tolerance **only** to the
    numeric value.
- Trailing zeros may still be ignored
    unless the tolerance note explicitly
    requires fixed formatting.

4. **Rankings / ordered lists** - Verify
    both the items and their order.
    **Exception for ties:** If multiple
    items have the exact same numerical
    value, any order
among those tied items is acceptable.
    Only evaluate rankings if the
    question actually asks for them.

5. **Label normalization / aliases** -
    Ignore differences in labels
    entirely. Do **not** consider
    variations in case, punctuation,
    spacing, apostrophes, typography, or
    shorthand forms when judging
    correctness. Label names are **not**
    used as a criterion for correctness;
    only the associated values or
    required information are evaluated.

6. **Formatting** - Ignore differences
    in wording, formatting, currency
    symbols, percent signs, or extra
    explanation. Judge factual
    correctness only.

7. **Scoring is binary** - Score **1**
    only if ALL required fields are
    correct. Score **0** if ANY required
    field is wrong or missing.

---

### Output Format

Reply in EXACTLY this format:

<reasoning>
Step 1: Identify which fields in the
    ground truth are actually requested
    by the question.
Step 2: Brief analysis of each required
    ground truth field vs. the solution.
    For numeric values, verify exact
    numeric equality after ignoring
    insignificant trailing zeros, with
    no rounding unless an explicit
    tolerance is provided. Apply label
    normalization for obvious aliases,
    and allow flexible ordering only for
    tied ranking values.
</reasoning>
<error>if Score is 0, list each
    incorrect or missing REQUIRED field
    and explain why it is wrong; if
    Score is 1, write "None"</error>
<score>0 or 1</score>
"""
```

E.3 Error Annotation Prompt

Error Annotation Prompt

```
You are an expert in error analysis for
    multi-turn, long-horizon data
    analysis benchmarks. Your task is to
    analyze why the agent answered each
    task incorrectly, and to distinguish
    ordinary data-analysis errors from
    long-horizon errors.

Read and use these files together:
- `code.py`: code generated by the agent.
- `ground_code.py`: reference solution
    code.
- `results_eval.json`: contains each
    task's question, context, reference
```

```
    answer, agent answer, agent
    trajectory, and evaluation process.
    The field `success` indicates
    whether the agent produced an
    answer; `judge.score` indicates
    whether the answer is correct.
- `task.ipynb`: benchmark notebook with
    task order, context, constraints,
    and reference solution logic.
- If necessary, inspect actual data
    files, but do not reinvent the task
    meaning. Treat `task.ipynb` and
    `ground_code.py` as authoritative.

Goal:
For each incorrect task, determine:
1. What went wrong;
2. Why it went wrong;
3. Whether the error was newly
    introduced in the current task or
    propagated from previous tasks;
4. Whether the error is an ordinary
    data-analysis error or a
    long-horizon / multi-round error.

Analyze tasks where `judge.score == 0`
    as incorrect. Do not rely on
    `success` alone: `success=True` can
    still be semantically wrong, and
    `success=False` often means the
    agent failed to produce a valid
    final answer.

Error types must be selected only from
    the following categories.

## Error Types

### 1. Statistical / Domain Reasoning
    Error

Errors caused by misunderstanding
    statistical concepts, metric
    semantics, dataset structure, field
    meanings, entity definitions,
    population scope, scale, or
    domain-specific analytical
    assumptions.

Use this type when the agent's mistake
    comes from an incorrect
    understanding of what a metric,
    field, entity, threshold,
    population, or domain concept means,
    even if the code runs.

Examples:
- Misunderstanding percentile vs
    quantile, percentile rank vs raw
    quantile cutoff, average-rank
    percentile vs min-rank percentile,
    rank vs score, z-score, residual,
    shrinkage, weighted mean, or support;
- Misunderstanding whether a threshold
    is absolute, relative, inclusive,
    exclusive, raw-scale,
    percentile-scale, rank-based, or
    population-relative;
- Computing a percentile, rank, cutoff,
    cap, median, or quantile over the
    wrong conceptual population, such as
    current candidates instead of the
    full working table, full data
    instead of current pool, or all
    players instead of side-specific
    players;
- Misunderstanding the meaning of
    exposure, coverage, risk, lift,
    residual, market lift, publisher
    support, neighbor support,
    contributor support, trend support,
    actionability, fragility,
    robustness, or reliability;
- Misunderstanding domain-specific
    fields such as season, region,
    station, district, surface, race
    distance, track condition, publisher
    family, work family, school group,
    cohort, market, or time window;
- Misunderstanding the unit of analysis,
    such as start vs horse, book vs work
    family, edition vs title, school vs
    district, station vs region,
    restaurant vs order, user vs
    interaction, player vs team, race vs
    season;
- Misunderstanding which population a
    metric is defined over, such as
    clean comparison starts, current
    warnings, reviewed group, candidate
    pool, active sample, strong
    finishers, reliable candidates, or
    final stable candidates;
- Misunderstanding the interpretation of
    a derived label, such as
    severe-stable, broader-dependent,
    actionable, repeated-stable,
    late-specific, fragile, robust,
    supported, current candidate, or
    reliable candidate;
- Misunderstanding the meaning of a
    comparison or diagnostic, such as
    treating a sensitivity check as a
    new default rule, treating a
    diagnostic support set as direct
    evidence, or interpreting a
    robustness count as another warning
    rule.

This type emphasizes that the agent
    misunderstood the statistical,
    semantic, or domain meaning of the
    task objects.

Important:
If the error involves the wrong metric
    population, wrong percentile/rank
    scale, wrong threshold semantics,
    wrong unit of analysis, or wrong
    interpretation of a derived label,
    include Statistical / Domain
    Reasoning Error in
    `all_error_types`, even if the task
    also involves instruction following,
    state management, or cascading.
```

```
For automated labeling, err on the side
    of including Statistical / Domain
    Reasoning Error in `all_error_types`
    when the mismatch involves
    percentile population, rank
    convention, score scale, cutoff
    population, unit of analysis, or
    derived-label semantics.

Do not use Statistical / Domain
    Reasoning Error when:
- The task explicitly stated a
    requirement and the agent simply
    omitted it without attempting it ->
    Instruction Following Error;
- The agent understood the concept but
    chose the wrong multi-step
    analytical route -> Planning Error;
- The agent understood the target
    concept and route but made a
    concrete code bug -> Coding /
    Implementation Error;
- The agent used the wrong concrete
    prior artifact, candidate pool,
    branch, or score table -> State
    Management Error;
- The current task is locally reasonable
    but inherits an already-wrong
    upstream artifact -> Cascade Error.

### 2. Planning Error

Errors in task decomposition, analytical
    strategy, method selection, or
    reasoning path.

Examples:
- Failing to recognize that the task
    requires constructing a candidate
    pool before ranking;
- Treating a stratified analysis problem
    as a global average problem;
- Choosing the wrong statistical test,
    modeling method, or analytical route;
- Failing to decompose a multi-stage
    analysis task in the correct order;
- Misjudging which intermediate results
    or comparison targets are needed for
    the current task;
- Performing exploratory analysis
    instead of completing the requested
    analytical workflow.

This type emphasizes incorrect
    analytical route or method design.

If the model identified the correct
    route but implemented it incorrectly
    in code, label it as Coding /
    Implementation Error.
If the model failed to follow steps or
    conditions explicitly stated in the
    current task, label it as
    Instruction Following Error.
If the model used the wrong prior state
    or branch, label it as State
    Management Error.
```

```
### 3. Instruction Following Error

Failure to follow requirements
    explicitly stated in the current
    task, including output format,
    processing method, formula,
    filtering condition, sorting rule,
    comparison target, or analysis step.

The key criterion:
The requirement is directly stated in
    the current task text or current
    context, and the model should have
    followed it in this turn.

Examples:
- The current task explicitly requires
    certain output fields, but the agent
    omits them;
- The current task explicitly requires a
    Boolean conclusion, but the agent
    does not output it;
- The current task explicitly requires a
    specific Top-K size, but the count
    is wrong;
- The current task explicitly specifies
    a sorting rule, but the agent does
    not follow it;
- The current task explicitly specifies
    decimal precision, but the agent
    does not follow it;
- The current task explicitly gives a
    formula, but the agent uses another
    formula;
- The current task explicitly requires a
    filtering condition, comparison
    target, or analysis step, but the
    agent does not apply it;
- The agent returns an empty answer,
    off-topic answer, exploratory notes,
    or asks what to do next instead of
    answering the task.

Do not label an error as Instruction
    Following Error merely because the
    final result violates the task.
    Decide whether the model ignored the
    requirement, implemented it
    incorrectly, planned the analysis
    incorrectly, misunderstood the
    metric/domain semantics, or used the
    wrong prior state.

If the current task explicitly states
    the population for a percentile,
    rank, cap, cutoff, or score and the
    agent uses a different population,
    this is both Instruction Following
    Error and Statistical / Domain
    Reasoning Error.

If a rule was established in a previous
    task or global convention but is not
    restated in the current task,
    forgetting it is Context Memory
    Error.

If the current task explicitly requires
```

```
    using a previous state, candidate
    pool, reviewed group, rollback
    branch, or previous artifact, but
    the model uses the wrong one,
    prioritize State Management Error.

If the current response is empty or
    off-topic, primary_error_type should
    usually be Instruction Following
    Error unless the trajectory clearly
    shows a runtime/code failure.

### 4. Coding / Implementation Error

Errors caused by concrete code-level
    implementation mistakes after the
    agent has otherwise identified the
    correct task requirement, analytical
    route, data scope, and inherited
    state.

Use this type only when the intended
    logic is substantially correct, but
    the code implementation is wrong.

Examples:
- Syntax error, runtime error, missing
    import, undefined variable, or
    invalid column reference;
- Code references the wrong DataFrame or
    column despite using the correct
    intended artifact;
- Merge/join is attempted with the
    correct tables but uses the wrong
    key or join type;
- Groupby/aggregation is attempted at
    the correct conceptual level but
    implemented with wrong grouping
    columns;
- Filter is attempted for the correct
    condition but implemented with an
    incorrect operator, boundary, or
    boolean combination;
- Rank/sort is attempted on the correct
    metric and population but
    implemented with wrong ascending
    direction, rank method, or
    tie-breaker;
- Deduplication is attempted for the
    correct entity level but implemented
    with the wrong subset/order;
- Text normalization or Unicode handling
    is implemented incorrectly;
- A specified formula is recognized and
    attempted, but transcribed
    incorrectly in code;
- Model or nearest-neighbor pipeline is
    conceptually appropriate, but has a
    concrete implementation bug such as
    failing to exclude self-neighbors,
    using the wrong fitted transformer,
    or applying train/test split
    incorrectly.

This type emphasizes: correct intent,
    wrong code.

Do not use Coding / Implementation Error
    when:
- The model misunderstood the
    statistical or domain concept ->
    Statistical / Domain Reasoning Error;
- The model chose the wrong analysis
    strategy or comparison design ->
    Planning Error;
- The model ignored a requirement
    explicitly stated in the current
    task -> Instruction Following Error;
- The model forgot a rule established
    earlier -> Context Memory Error;
- The model used the wrong candidate
    pool, state, branch, score table, or
    previous artifact -> State
    Management Error;
- The current task is locally reasonable
    but inherits an already-wrong
    upstream artifact -> Cascade Error.

Do not label a task as Coding /
    Implementation Error merely because
    `code.py` differs from
    `ground_code.py`. First identify
    whether the difference is caused by
    wrong intent, wrong state, wrong
    metric semantics, or wrong
    implementation.

### 5. Context Memory Error

Failure to remember long-range rules,
    global conventions, or default
    specifications established earlier
    but not explicitly restated in the
    current task.

The key criterion:
The requirement is not newly given in
    the current task. It was established
    in an earlier task, global
    instruction, or earlier context, and
    the current turn requires
    long-horizon memory to continue
    following it.

Examples:
- Forgetting previously specified
    decimal precision;
- Forgetting a default sorting
    convention, such as "results are
    listed from strongest to weakest
    unless otherwise specified";
- Forgetting previously specified output
    field requirements;
- Forgetting a previously defined
    processing method, formula,
    filtering condition, or analysis
    step;
- Forgetting a previously defined naming
    convention, grouping rule, default
    filtering rule, or percentile
    convention;
- Forgetting the rule meaning of a
    previously defined reviewed group,
    candidate definition, or analysis
    convention.
```

```
This type emphasizes loss of rule memory.

Important boundary:
Context Memory Error concerns forgetting
    rules, conventions, definitions, or
    default specifications.
State Management Error concerns using
    the wrong concrete intermediate
    state, candidate pool, score table,
    branch, or artifact.
Statistical / Domain Reasoning Error
    concerns misunderstanding the
    meaning of a metric, population,
    threshold, or domain concept.

Do not use Context Memory Error as
    primary merely because an upstream
    score drifted. If the current task
    correctly uses an already-wrong
    upstream score/table/set,
    primary_error_type should be Cascade
    Error.

If the current task explicitly restates
    the requirement and the model still
    fails to follow it, prioritize
    Instruction Following Error.

### 6. State Management Error

Failure to correctly inherit, update,
    roll back, isolate, or reuse
    intermediate state from previous
    tasks.

Examples:
- Using the wrong current candidate pool;
- Using the wrong reviewed group, final
    set, lifted slice, or long-risk set;
- Using the wrong adjusted score,
    current quality, folded-market
    score, contributor-adjusted score,
    or other intermediate column;
- Continuing to use the current branch
    when the task requires the rollback
    state;
- Comparing the current result to raw
    data when the task requires
    comparing pre-rollback and
    post-rollback states;
- Letting a local perturbation leak into
    later default state;
- Continuing to use an old state when an
    updated definition should be
    inherited;
- Mixing separated-market treatment with
    merged-market treatment;
- Using the wrong previous model output
    or previous filtering result;
- Selecting the wrong saved artifact
    among several previous views, such
    as earliest formula, current score,
    middle-stage score, diagnostic
    score, or final score.

This type emphasizes using, inheriting,
    updating, rolling back, or isolating
    dynamic intermediate state
    incorrectly.

If the problematic object is a specific
    set, table, score, branch, candidate
    pool, model output, or intermediate
    result, prioritize State Management
    Error over Context Memory Error.

If the intended prior artifact exists
    and the current task selects the
    wrong one, this is State Management
    Error, not Cascade Error.

### 7. Cascade Error

The current task is locally reasonable
    under the inherited intermediate
    result, state, or semantic
    assumption, but that inherited
    artifact was already wrong due to an
    earlier task, causing the current
    answer to be wrong even without a
    new independent error.

Use Cascade Error only when:
- The current task has no obvious new
    independent error;
- The current task actually uses an
    incorrect upstream artifact, such as
    a wrong candidate pool, score,
    label, baseline, mapping, model
    output, or intermediate result;
- If the upstream artifact were
    corrected, the current answer would
    likely become correct or close to
    correct.

Do not mark Cascade Error merely because
    an earlier task was wrong.

Use Cascade Error as
    `primary_error_type` when the
    current task is locally reasonable
    and the main reason it is wrong is
    that it inherited an already-wrong
    artifact. In that case, do not use
    Context Memory Error or State
    Management Error as primary unless
    the current task independently
    forgot a rule or selected the wrong
    artifact.

Do not output `all_error_types =
    ["Cascade Error"]` unless the
    current task's procedure, metric
    semantics, population choice, state
    selection, and output format are all
    locally correct under the inherited
    artifact.

Examples:
- The current task correctly sums a
    score share, but the inherited score
    values from an earlier task are
    wrong.
- The current task correctly ranks the
    current warning set, but the current
    warning set was already wrong.
```

```
- The current task correctly decomposes
    the selected candidate, but the
    selected candidate came from an
    earlier wrong ranking.
- The current task correctly compares
    groups, but the group labels were
    already wrong upstream.

If the current task introduces a clear
    new error:
- `primary_error_type` should be the
    current-turn error type;
- `all_error_types` may include Cascade
    Error;
- `upstream_error_task_id` should
    identify the relevant upstream task.

If the current response is empty,
    off-topic, exploratory, or fails to
    answer the current task's explicit
    request, prioritize Instruction
    Following Error and do not mark
    Cascade Error unless the response
    clearly uses a specific incorrect
    inherited artifact.

## Annotation Rules

For each incorrect task:
- `primary_error_type`: choose exactly
    one type that most directly explains
    the current incorrect answer.
- `all_error_types`: include all
    relevant error types. Include
    Cascade Error only if the current
    task actually depends on an
    incorrect upstream artifact.
- `upstream_error_task_id`: use `null`
    if there is no cascade. If Cascade
    Error appears in
    `primary_error_type` or
    `all_error_types`, provide the
    upstream task id; use an array if
    multiple tasks contributed.
- If the current task has a new
    independent error, do not use
    Cascade Error as the primary type.
- If the current task is locally correct
    or mostly reasonable but uses an
    already-wrong upstream artifact, use
    Cascade Error as primary.
- If the correct prior artifact exists
    but the current task selects,
    updates, rolls back, or reuses the
    wrong artifact, use State Management
    Error as primary.
- If the current task uses the intended
    prior artifact correctly but that
    artifact was already wrong, use
    Cascade Error as primary.
- If the agent used the wrong data
    scope, candidate pool, prior state,
    branch, or intermediate table,
    prefer State Management Error over
    Coding / Implementation Error.
- If the agent used a different formula
    because it ignored the formula
    explicitly stated in the current
    task, prefer Instruction Following
    Error.
- If the agent used a different formula
    because it misunderstood the metric
    meaning, scale, threshold, or
    population, prefer Statistical /
    Domain Reasoning Error.
- If the agent selected the wrong
    analytical route before coding,
    prefer Planning Error.
- If the error involves percentile/rank
    population, cutoff population,
    metric scale, threshold semantics,
    entity granularity, or unit of
    analysis, include Statistical /
    Domain Reasoning Error in
    `all_error_types`.
- Before assigning Cascade Error,
    explicitly check whether the current
    task itself introduced any
    independent error.
- If the current task explicitly states
    a formula, population, percentile
    ladder, candidate pool, ranking
    scope, comparison target, or output
    requirement, and the agent violates
    it, then `all_error_types` must
    include Instruction Following Error,
    even if the task also inherits an
    upstream error.
- If the current task involves
    percentile, rank, cutoff, cap,
    threshold, population semantics and
    the agent uses the wrong population,
    scale, rank convention, or threshold
    interpretation, then
    `all_error_types` must include
    Statistical / Domain Reasoning Error.
- Cascade Error alone is allowed only
    when the current task has no
    independent mismatch with the
    current task instruction, metric
    semantics, population choice, state
    selection, or output format.
- If the current task both inherits a
    wrong upstream artifact and violates
    a current-task requirement,
    `all_error_types` must include both
    Cascade Error and the current-task
    error type.
- If the current task uses a wrong
    score, label, candidate pool, or
    table that was produced by a
    previous incorrect task,
    `all_error_types` must include
    Cascade Error and
    `upstream_error_task_id` must name
    that previous task.
- If the current task selects the wrong
    prior artifact among multiple
    available prior artifacts,
    `all_error_types` must include State
    Management Error. If the selected
    artifact was also already wrong,
    `all_error_types` may also include
    Cascade Error.
- Do not let an empty or off-topic
    answer in an upstream task
```

```
    automatically become the root cause
    for later tasks. For downstream
    tasks, identify the actual wrong
    artifact being inherited.
- Do not truncate any field with
    ellipses such as "...". Write
    complete concise sentences.

## Analysis Steps

1. Read `results_eval.json` first and
    identify all tasks with `judge.score
    == 0`.
2. For each incorrect task, inspect the
    current question, context, reference
    answer, agent answer, trajectory,
    and evaluation reasoning.
3. Compare `ground_code.py` and
    `code.py` to determine whether the
    discrepancy comes from output
    format, filtering scope, sorting,
    formula, join, aggregation, modeling
    pipeline, state inheritance, context
    memory, statistical/domain
    semantics, or upstream cascading.
4. If needed, review previous tasks in
    `task.ipynb` to determine whether a
    missed condition was stated in the
    current task or only established
    earlier.
5. If the task depends on a previous
    result, check whether the previous
    artifact was already wrong and
    whether the current task actually
    used it.
6. Explain the concrete mechanism
    causing the mismatch. Do not infer
    only from final numerical
    differences.
7. When multiple error types overlap,
    choose the most direct current cause
    as `primary_error_type` and include
    other relevant types in
    `all_error_types`.

## Required Output

Return a valid JSON object only. Do not
    output Markdown or explanatory prose.

Use this format:

{
  "summary": {
    "total_incorrect_tasks": 0,
    "primary_error_type_counts": {
      "Statistical / Domain Reasoning
    Error": 0,
      "Planning Error": 0,
      "Instruction Following Error": 0,
      "Coding / Implementation Error": 0,
      "Context Memory Error": 0,
      "State Management Error": 0,
      "Cascade Error": 0
    },
    "all_error_type_counts": {
      "Statistical / Domain Reasoning
    Error": 0,
      "Planning Error": 0,
      "Instruction Following Error": 0,
      "Coding / Implementation Error": 0,
      "Context Memory Error": 0,
      "State Management Error": 0,
      "Cascade Error": 0
    },
    "cascade_error_count": 0,
    "main_failure_patterns": [
      "Main failure pattern 1",
      "Main failure pattern 2"
    ]
  },
  "task_error_analysis": [
    {
      "task_id": 1,
      "primary_error_type": "one of the
    allowed error types",
      "all_error_types": ["one or more
    allowed error types"],
      "upstream_error_task_id": null,
      "error_summary": "Complete concise
    explanation of why the task is
    wrong.",
      "evidence": "One concise sentence
    comparing the task/reference
    behavior with the agent behavior.",
      "confidence": "high / medium / low"
    }
  ]
}

Save the final JSON object to a file
    named `error_ana.json` in the
    current working directory.
```